



MKS SERVO 42ES/57ES Closed-Loop Stepper Motor

CAN User Manual V1.0.1

Note: This manual corresponds to firmware version V 1.0.1 .

MKS SERV042ES /57ES _CAN Version Notes			
Instruction manual version	content	Firmware version	date
V1.0.0	First release version	V1.0.0	Dec-2024
V1.0.1	1. Fixed some bugs	V1.0.1	Nov-2025



Catalog

Part 1.	Product Overview.....	5
1.1	Product Introduction.....	5
1.2	Features.....	5
1.3	Product Parameters.....	6
1.4	Interface Description.....	7
1.5	Port Function Description	9
1.6	Indicator Light Status Description Table.....	10
Part 2.	Wiring method	11
2.1	Motor wiring method.....	11
2.2	Pulse control wiring method	12
2.3	CAN/RS485 wiring method	13
2.4	External switch wiring method.....	15
Part 3.	CAN frame format description.....	17
Part 4.	General Instructions.....	18
4.1	Calibrate motor instructions	18
4.2	Read-only parameter instructions.....	19
4.2.1	Reading the value of a multi-turn encoder	19
4.2.2	Read the cumulative multi-turn encoder value	20
4.2.3	Read the real-time speed of the motor.....	20
4.2.4	Read the number of received pulses.....	21
4.2.5	Read motor alarm status.....	21
4.2.6	Read position angle error	22
4.2.7	Read motor enable status	22
4.2.8	Read the motor return to zero status.....	23
4.2.9	Automatically return read-only parameters at regular intervals	24
4.2.10	Read a single configuration parameter	25
4.2.11	Read version information	26
4.2.12	Read /Write User-defined ID.....	27
4.3	Write-only parameter command	28
4.3.1	Set working mode	28
4.3.2	Set operating current	29
4.3.3	Set subdivision	29
4.3.4	Set the active level of the En pin.....	30
4.3.5	Set the motor rotation direction	30
4.3.6	Set pulse delay	31
4.3.7	Set bit rate	31
4.3.8	Set slave ID	32
4.3.9	Configure slave response method.....	33
4.3.10	Configure whether to lock the axis when starting in bus mode.....	34
4.3.11	Set group ID.....	35
4.3.12	Set heartbeat protection time.....	36
4.3.13	Set position to reach threshold	36



4.3.14	Set no-verification mode	37
4.4	Stall protection instructions	38
4.4.1	Set stall protection and over-tolerance settings.	38
4.4.2	Read motor stall status.....	39
4.4.3	Release the motor from stall state	39
4.5	Save /Restore Parameters and Reset Instructions	40
4.5.1	Save parameter command	40
4.5.2	Restore factory settings.....	40
4.5.3	Reset and restart the motor	41
Part 5.	IO Port Operation Instructions	42
5.1	Read I/O port status	42
5.2	Write data to IO port	43
5.3	IO port pulse frequency division output	44
Part 6.	Motor return to zero instructions	45
6.1	Explanation of the method of returning to zero at the origin.....	45
6.2	Explanation of coordinate zeroing method	46
6.3	Set parameters related to zero return.....	47
6.3.1	Set the zero-return mode, direction, and speed parameters.....	47
6.3.2	Set the zero-return torque and origin offset parameters.....	48
6.3.3	Configure the zero-return trigger method and timeout parameters.	49
6.3.4	Setting Zero Point Instructions	49
6.3.5	Execute the zero-return instruction	50
6.4	Switch return to zero configuration example.....	51
6.4.1	Example of a switch command triggering a return to zero	51
6.4.2	Example of automatic return to zero upon power-on.....	52
6.5	Example of mechanical limit switch homing configuration.....	53
6.5.1	Example of mechanical limit command triggering homing.....	53
6.5.2	Example of automatic zeroing upon power- on of mechanical limit switch ...	54
6.6	Single-lap zero-return configuration example.....	55
6.6.1	Example of single-lap command triggering zero return	55
6.6.2	Example of automatic zero return upon power-on in a single revolution	56
6.7	Example of coordinate zeroing.....	57
Part 7.	Left and right limit switch instructions.....	58
7.1	Set limit switch parameters.....	58
7.2	Example of Limit switch configuration	59
7.3	Example of Left limit switch test	60
7.4	Example of Right limit switch test	60
Part 8.	Bus control mode parameter description	61
8.1	Description the parameters of speed and acceleration	61
8.2	Bus control general instructions.....	63
8.2.1	Read motor operating status.....	63
8.2.2	Set motor enable state.....	64
8.2.3	Emergency stop command for motor	64
Part 9.	Speed Control Mode Description	65



9.1	Speed control mode operation instructions	65
9.2	stop command	67
9.3	Set automatic start command upon power-on	68
9.4	Example of Speed mode running	69
9.5	Example of Automatic run mode	70
Part 10.	Position control mode description	71
10.1	Relative motion according to pulse number	71
10.1.1	Relative pulse operation command	71
10.1.2	Stop command	72
10.1.3	Example of Relative Pulse Operation	73
10.2	Absolute motion based on pulse count	74
10.2.1	Absolute Pulse Operation Command	74
10.2.2	Stop command	76
10.2.3	Example of Absolute Pulse Operation	77
10.3	Relative motion according to coordinate values	78
10.3.1	Relative coordinate running instructions	78
10.3.2	Stop command	80
10.3.3	Example of relative coordinate operation	81
10.4	Absolute motion based on coordinate values	82
10.4.1	absolute coordinate execution command	82
10.4.2	Stop command	84
10.4.3	Example of absolute coordinates operation	85
10.4.4	Example of Real-time updates operation	86
Part 11.	Multi- motor synchronous motion description	87
11.1	synchronous motion mode 1 - Broadcast	87
11.2	synchronous motion mode 2 - grouping method	88
11.3	synchronous motion mode 3 - Synchronization mark	88
11.3.1	Command to enable/disable synchronization function	88
11.3.2	Synchronous execution instructions	89
11.3.3	Example of multi-motor synchronous operation	90
Part 12.	CANGAROO usage examples	91
12.1	Motor parameter configuration	91
12.2	cangaroo parameter configuration	91
12.3	Example of reading encoder values	94
Part 13.	Frequently Asked Questions and Precautions	95
13.1	Precautions	95
13.2	Frequently Asked Questions	96
Part 14.	contact us	96



Part 1. Product Overview

1.1 Product Introduction

The MKS SERVO42ES/57ES _RS485 closed-loop stepper motor is a product independently developed by the Maker Base to meet market demands and meet industrial-grade standards . It features a pulse interface and an RS485 interface, a built-in high-efficiency FOC vector algorithm , and a high-precision encoder . Through position feedback, it effectively prevents step loss. It is suitable for applications such as small robotic arms , 3D printers , engraving machines, writing machines , automation products , and e-sports competitions .

1.2 Features

1. Supports 6 operating modes : pulse interface (pulse-pulse, pulse-direction mode) , serial interface (open-loop, closed-loop mode);
2. High-performance FOC vector control algorithm, torque , speed, and position three-loop control ;
3. It supports curved acceleration and deceleration, resulting in smoother motor start-up and stopping;
4. Supports both relative and absolute position control modes;
5. Built-in 256-step subdivision interpolation algorithm , ultra- quiet motor operation , ultra-low vibration;
6. Maximum input pulse frequency : 300 kHz; maximum speed: 3000 RPM.
7. Motor angle information is updated in real time (motor enabled or disabled) ;
8. Supports MODBUS -RTU communication protocol;
9. It has built-in stall protection function ;
10. Quick factory reset function;
11. High-speed performance is stable, operation is smooth and vibration-free, and emergency stop is possible ;
12. The integrated aluminum alloy casing provides effective heat dissipation and ensures more stable continuous high-current operation of the motor .
13. Provides an open-source host computer and STM32/ Arduion usage examples.
14. Supports left and right limit functions.
15. Onboard industrial-grade high-precision magnetic encoder ;
16. Onboard high - power MOSFET , 60V/56A;
17. It features an onboard RS-485 interface, 256 slave IDes , and supports broadcast and packet IDes .
18. Industrial-grade design and selection, stable and reliable;
19. Operating voltage: 20V~48V ;

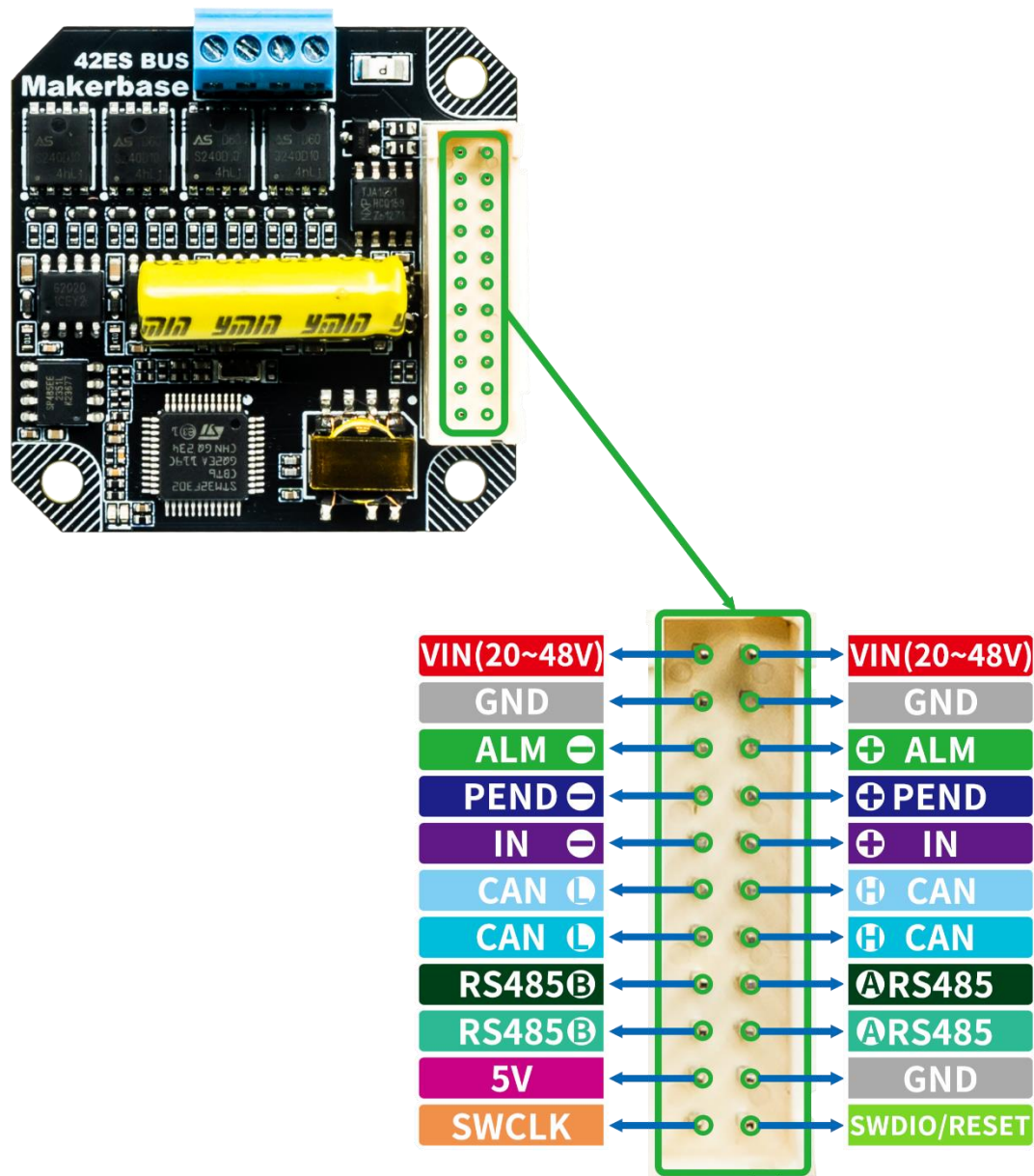


1.3 Product Parameters

Product Parameters		
Motherboard Model	MKS SERV042ES V1.0	MKS SERV057ES V1.0
MCU main controller	STM32F302CBT6	STM32F302RBT6
MOSFET	HYG090ND06LS1C2	
Magnetic encoder	MT6826	
CAN transceiver	TJA1051T	
Operating voltage	20-48V	
Operating current	0-3000mA	0-5200mA
Closed-loop feedback frequency	Torque ring 20KHz	
	Speed loop 10kHz	
	Position ring 10KHz	
Maximum speed	3000RPM+	
Subdivision support	2/4/8/16/32/64/128/5/10/20/25/40/50/100/200/256	
Wiring method	Common positive, common negative, difference	
Input signal level	3.3V-24V	
Pulse signal frequency	Up to 300kHz	
CAN interface rate	125K/250K/500K/1M	
CAN interface ID	1 broadcast ID, 2047 slave IDes	

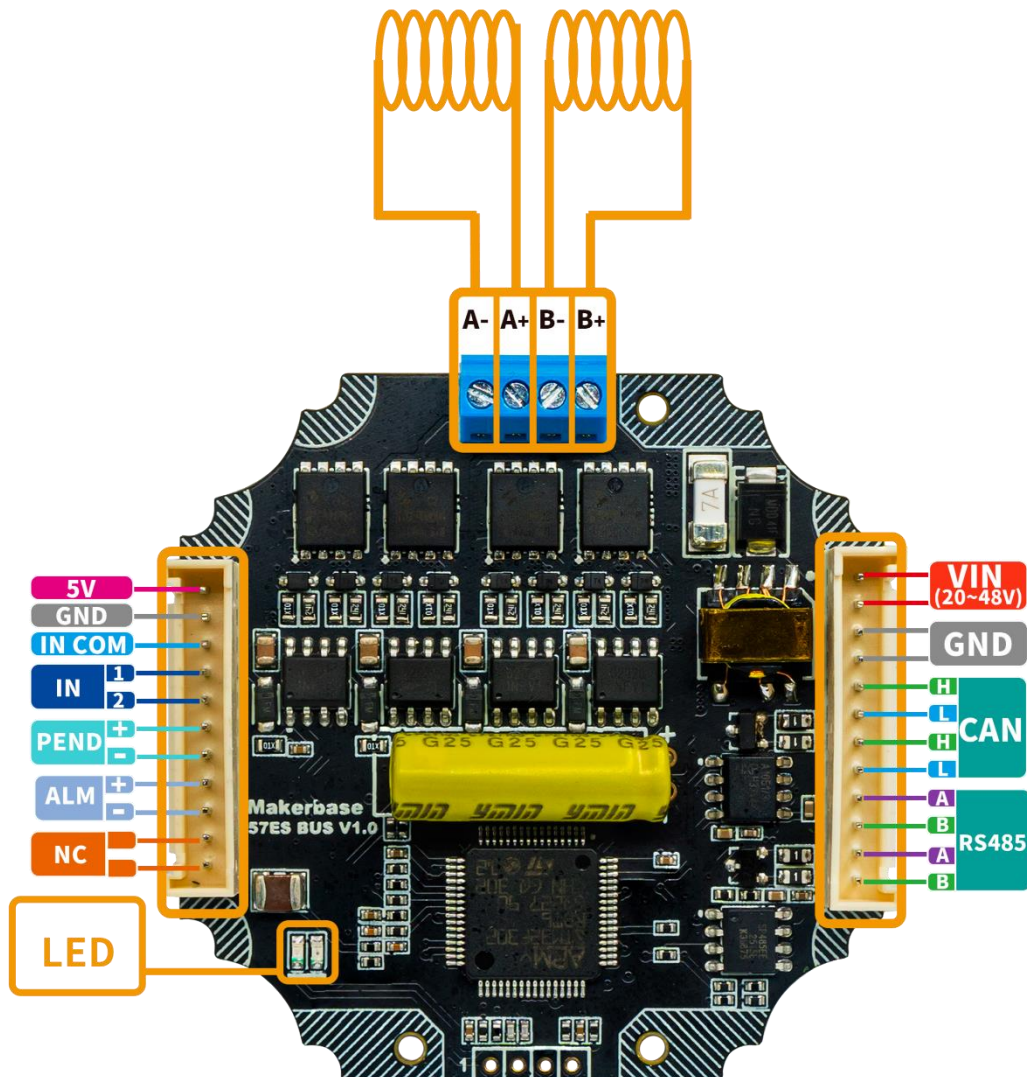
1.4 Interface Description

① SERV042ES Interface Description



1. Input power supply 20V~48V
2. 5V is the output power (5V, 100mA).
3. ALM is an alarm signal (the optocoupler closes when an alarm occurs).
4. PEND is the positioning signal (after positioning, the optocoupler closes).
5. IN is the input signal, IN is the zero-return switch or left limit switch, EN can be remapped to the right limit switch. For details on remapping, please refer to the limit switch setting instructions in Part 4 (when EN is mapped to the right limit switch, the pulse control mode is disabled).
6. Two sets of CAN interfaces are connected in parallel; you can connect either set.
7. Two sets of RS485 interfaces are connected in parallel; you can connect either set.

② SERV057ES CAN Interface Description



1. Input power supply 20V~48V
2. 5V is the output power (5V, 100mA).
3. ALM is an alarm signal (the optocoupler closes when an alarm occurs).
4. PEND is the positioning signal (after positioning, the optocoupler closes).
5. IN COM is the common port of IN_1 and IN_2 (can be connected in either common cathode or common anode configuration).
6. IN_1 and IN_2 are input signals. IN_1 is the zero-return switch or left limit switch, and IN_2 is the right limit switch.
7. Two sets of CAN interfaces are connected in parallel; you can connect either set.
8. Two sets of RS485 interfaces are connected in parallel; you can connect either set.



1.5 Port Function Description

Port Function Table			
PIN	I/O	Function	Description
STP+ (CW+)	Input	Pulse signal (or CW pulse) positive	(1) Optocoupler input, falling edge trigger. The pulse goes one step when the pulse goes from high to low. (2) Positive voltage: 3.3 - 28V. (3) Negative voltage: High level 3.3 - 28V, Low level 0 - 0.5V. (4) Maximum input frequency 400kHz, pulse duration greater than 2.5us. (5) Single pulse (Pulse + Dir) or double pulse (CW + CCW) is set by the command.
STP- (CW-)		Pulse signal (or CW pulse) negative	
DIR+ (CCW+)		Direction signal (or CCW pulse) positive	
DIR- (CCW-)		Direction signal (or CCW pulse) negative	
EN+		Enable signal positive	(1) Optocoupler input, low level effective. When ineffective, the motor will be released while clearing the alarm signal. (2) Positive voltage: 3.3 - 28V. (3) Negative voltage: High level 3.3 - 28V, Low level 0 - 0.5V.
EN-		Enable signal negative	
ALM+	Output	Alarm signal positive	(1) When an alarm occurs, the optocoupler output is on. (2) Maximum voltage +35V, maximum current 50mA.
ALM-		Alarm signal negative	
PEND+		In Position signal positive	(1) When the drive has finished a given pulse, the optocoupler output is on. (2) Maximum voltage +35V, maximum current 50mA.
PEND-		In Position signal negative	
A-	Output	Motor phase A-	Connected to motor phase line Note: If the phase sequence is not connected accordingly, the motor will alarm.
A+		Motor phase A+	
B-		Motor phase B-	
B+		Motor phase B+	
VDC	Input	Drive power supply positive	Operating Voltage +20 - 48V
GND		Drive power supply negative	



1.6 Indicator Light Status Description Table

LED Status	
Green stays on	Motor Running
Green flash	Motor Stop
1 Green + 1 Red	Over Current
1 Green + 2 Red	Open-Phase
1 Green + 3 Red	Supply Voltage High
1 Green + 4 Red	Supply Voltage Low
1 Green + 5 Red	Position Error
1 Green + 6 Red	Encoder Error

Tip: Please make sure the motor phase line is connected correctly, otherwise there will be a tracking error alarm after receiving the pulse (5 red and 1 green).

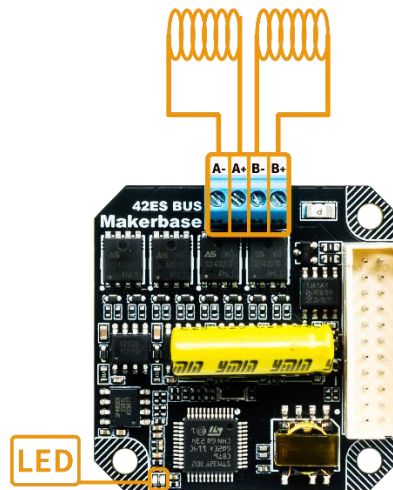
Part 2. Wiring method

2.1 Motor wiring method

Note: The internal resistance of the motor should be less than 10 ohms.

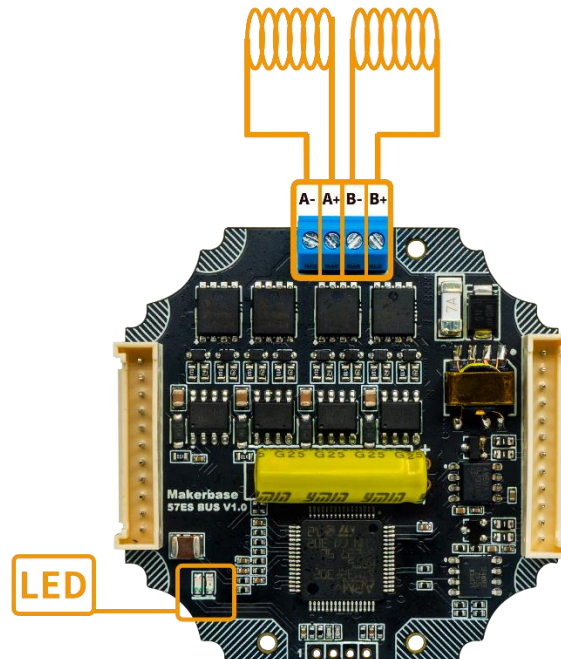
A+ and A- to one phase wire of the motor, and connect B+ and B- to the other phase wire of the motor.

① SERVO42ES Wiring Method



Note: Please ensure the motor phase wires are connected correctly; otherwise, a tracking error exceeding tolerance alarm (5 red, 1 green) will occur upon receiving the pulse.

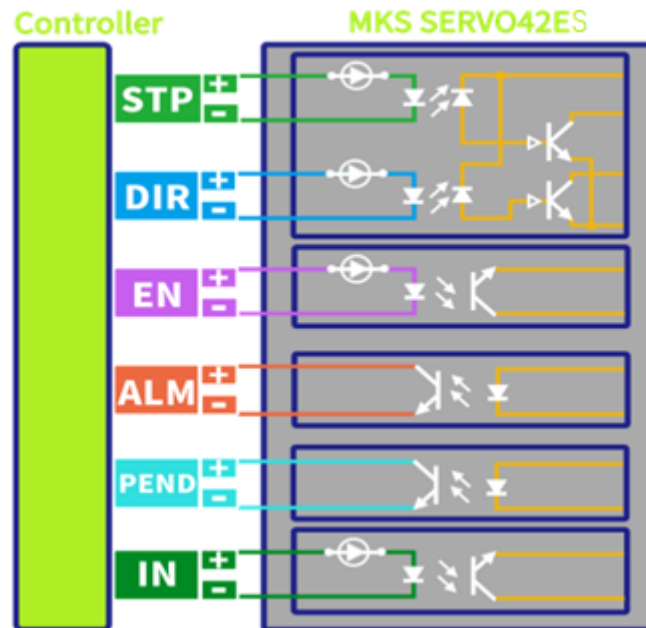
② SERVO57ES Wiring Method



Note: Please ensure the motor phase wires are connected correctly; otherwise, a tracking error exceeding tolerance alarm (5 red, 1 green) will occur upon receiving the pulse.

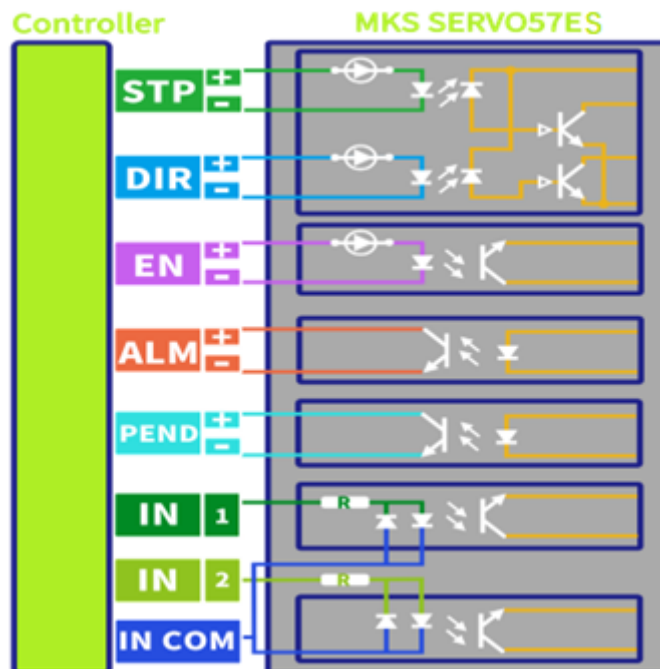
2.2 Pulse control wiring method

① SERV042ES Wiring Method



Note: All input ports have a built-in 10mA current limiter, allowing direct input of 3.3V-24V signals without the need for external current-limiting resistors.

② SERV057ES Wiring Method



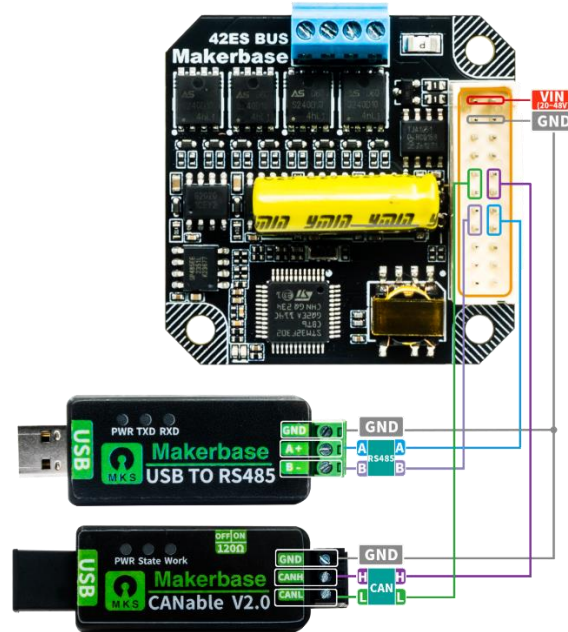
Note: All input ports have a built-in 10mA current limiter, allowing direct input of 3.3V-24V signals without the need for external current-limiting resistors.

2.3 CAN/RS485 wiring method

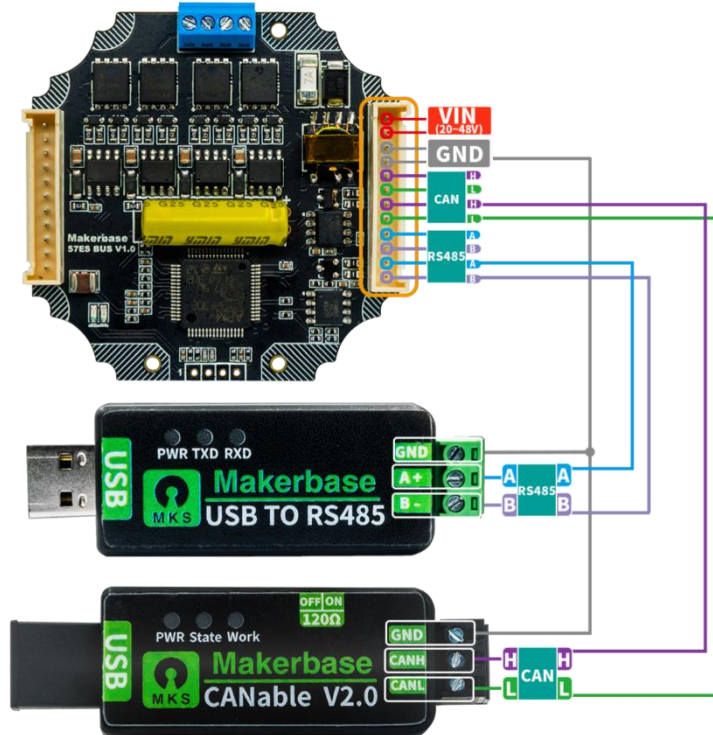
1. standalone communication wiring

Note: To reduce bus interference, the host computer and the motor must share a common ground, and CAN signals should be transmitted using shielded twisted-pair cable.

① MKS SERVO42ES standalone wiring

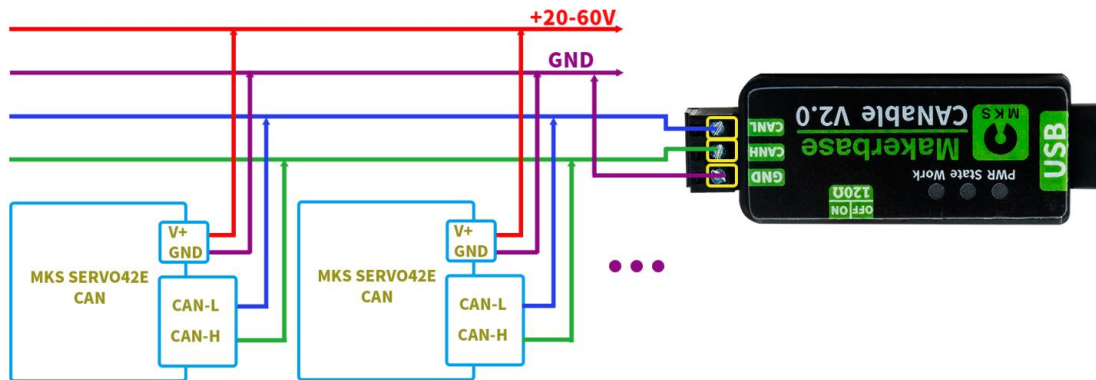


② MKS SERVO57ES standalone wiring

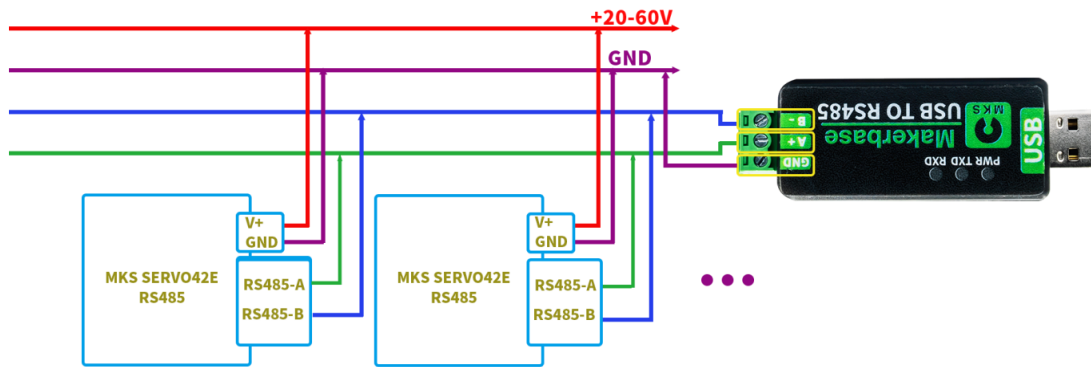


2. multi-machine communication wiring

① CAN Multi-machine Wiring



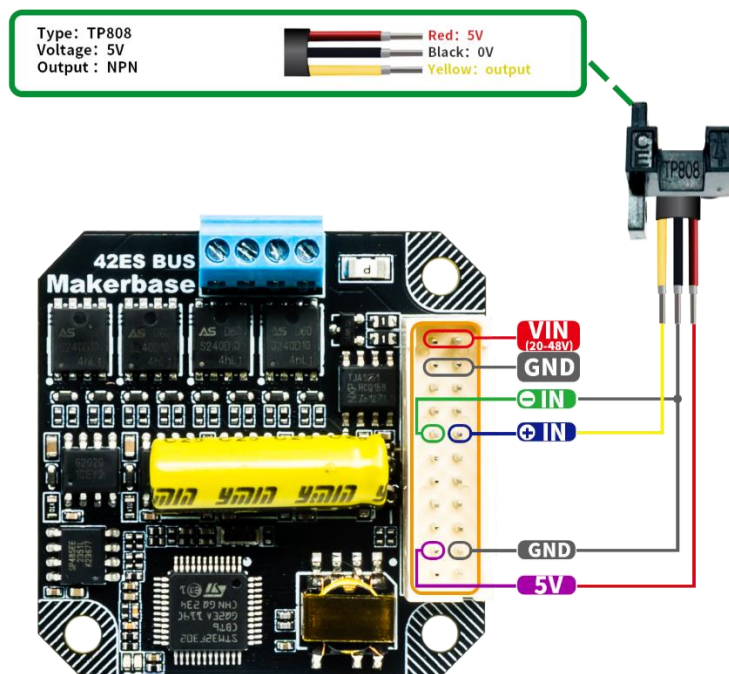
② RS485 Multi-unit Wiring



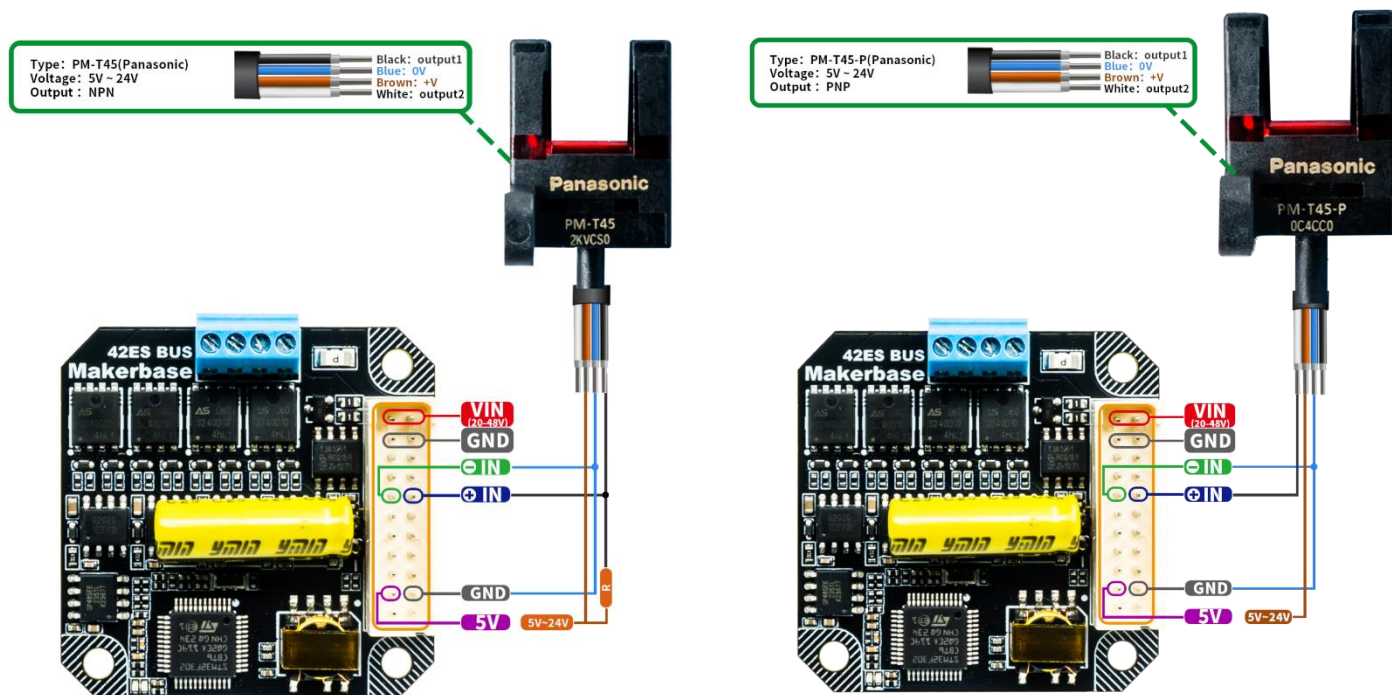
2.4 External switch wiring method

① Connection method of SERV042ES

(1) TP808 wiring

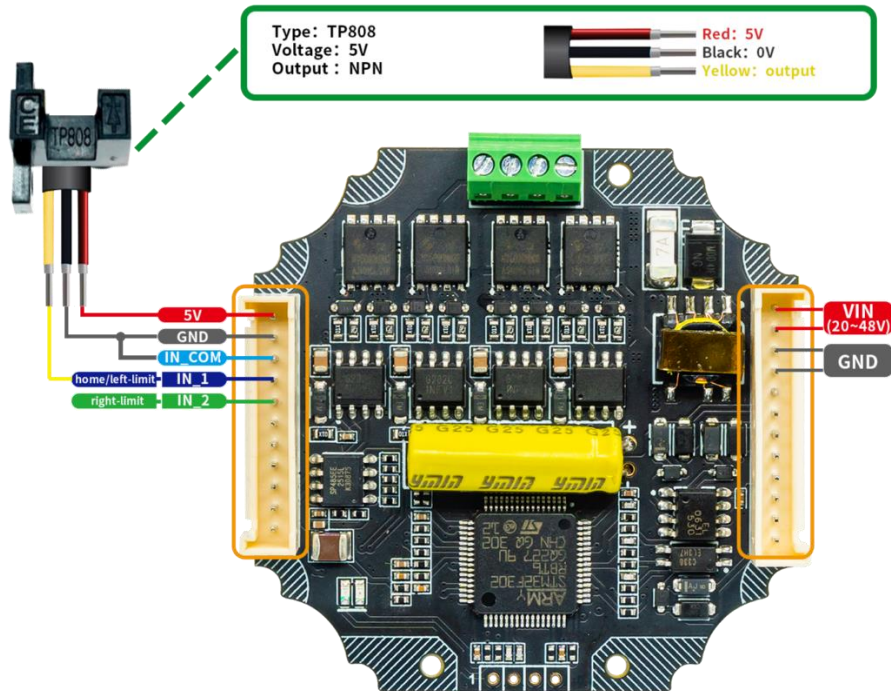


(2) PM-T45 and PM-T45-P wiring

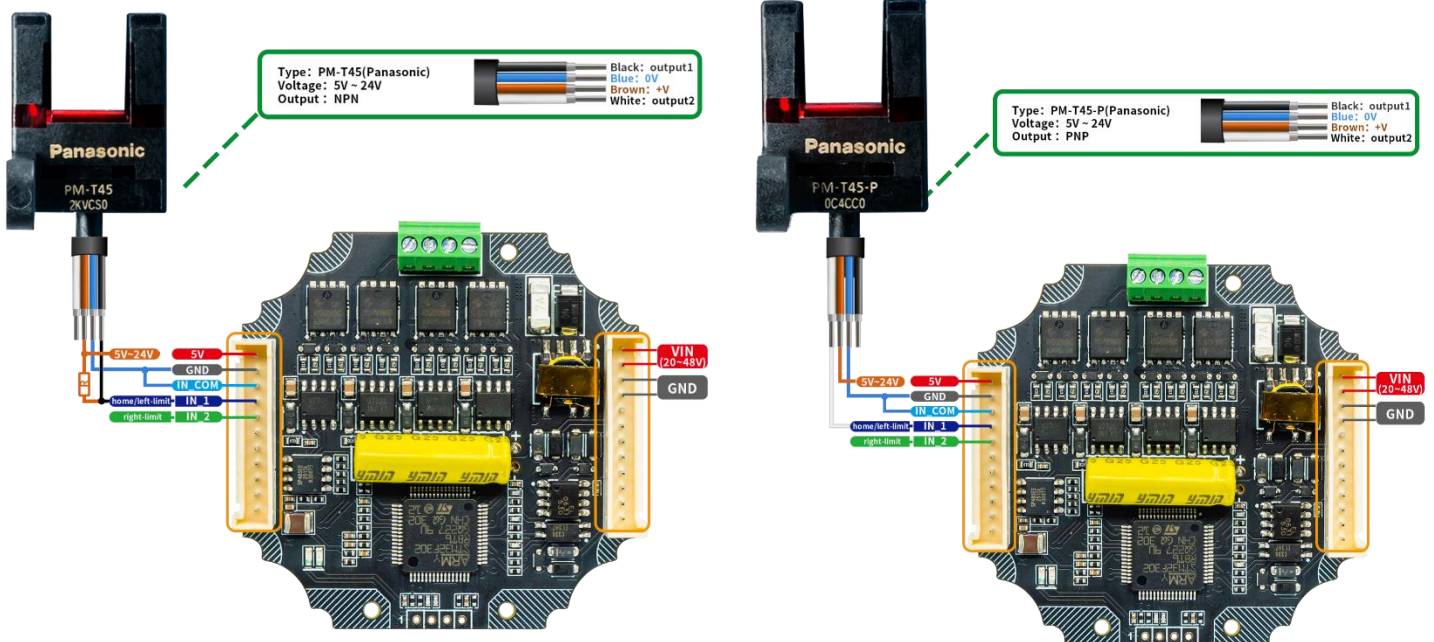


② Connection method of SERV057ES

(1) TP808 wiring



(2) PM-T45 and PM-T45-P wiring





Part 3. CAN frame format description

The message uses a standard frame, and the maximum length of the data field is Byte8.

Downlink frames (PC → SERV0xxES)					
CAN ID	...	DLC	Byte1	Byte2...Byte(n-1)	Byte n (n ≤ 8)
ID		DLC(n)	code	data	CRC

Uplink frames (PC ← SERV0xxES)					
CAN ID	...	DLC	Byte1	Byte2...Byte(n-1)	Byte n (n ≤ 8)
ID		DLC(n)	code	data	CRC

1. The CAN ID range is 00~2047. (default is 01) .
00 is the broadcast CAN ID;
01~10 can be set in the CanID option of the display menu;
greater than 10 need to be set by serial commands.
2. The code (code) executes the corresponding command.
for example, 0x80 executes the calibration command.
3. The Check code is CHECKSUM 8bit

For example, the command to read encoder values:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	byte2
01		2	30	CRC

$$\text{CRC} = (0x01 + 0x30) \& 0xFF = 0x31 \& 0xFF = 0x31$$

Note: When sending commands using a broadcast CAN ID, the slave device will not respond.



Part 4. General Instructions

Note 1: The default ID for this chapter is 01 .

Note 2: When sending commands using a broadcast ID, the slave device will not respond.

Note 3: All write-only instructions are not saved immediately after being set; the 60H instruction must be executed to save them all at once.

4.1 Calibrate motor instructions

Note: The complete machines sold by the manufacturer are already calibrated at the factory and do not require recalibration.

Calibration can compensate for and correct nonlinearities caused by magnet installation deviations.

If the motor phase wires are wired according to the factory default connection, no calibration is required. Although it can be used, the motor performance may be poor.

To ensure the motor performs optimally, or if issues such as excessive motor vibration or inconsistent current (sound) in both forward and reverse directions occur, the motor can be recalibrated.

The motor must be unloaded before calibration and must not be under any load.

During calibration, the motor rotates at a speed of 600 RPM for more than 20 revolutions.

If calibration is successful, the green light will flash rapidly. Power off and restart the motor before it can be used normally.

If calibration fails, the red light will flash rapidly. You need to power off and restart the motor, and then calibrate again.

Code 0x80

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	80H	01	CRC(81)

No data was returned.

After receiving the command, the motor will automatically restart and enter calibration mode. You need to wait about 10 seconds to observe the status of the LED lights and judge the calibration result.

4.2 Read-only parameter instructions

4.2.1 Reading the value of a multi-turn encoder

It can read the motor rotation range recorded by the encoder after power-on.

Note: Encoder single-turn value range is 0~0x4000

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01	...	2	30H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)							
ID	...	DLC	Byte1	Byte2...Byte5	Byte6	Byte7	Byte8
01	...	8	code	Carry value	Encoder value		Checksum
			30H	carry (int32_t)	value(uint16_t)		CRC

carry: the carry value of the encoder.

value: the current value of the encoder.(range 0~0x3FFF)

When value is greater than 0x3FFF, carry +=1.

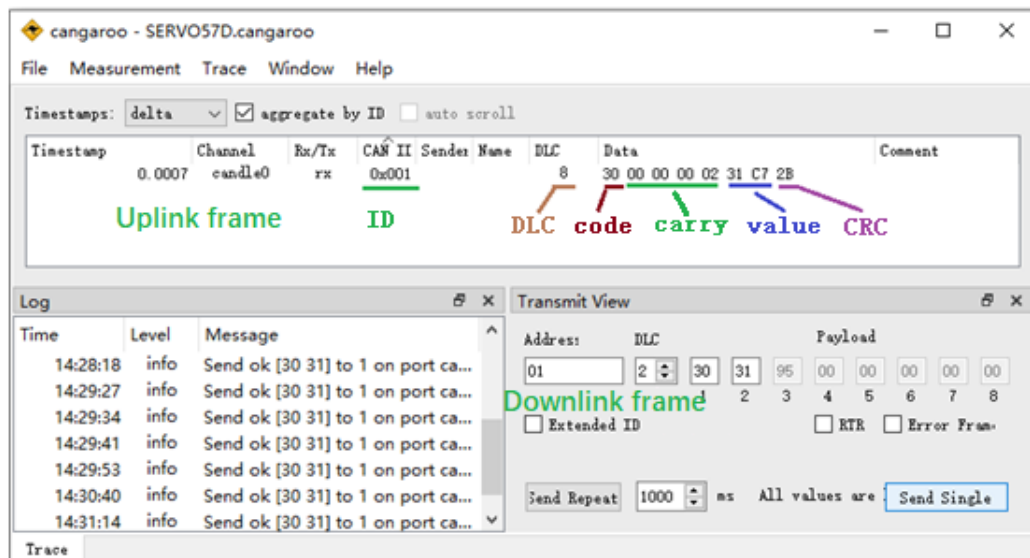
When Value is less than 0, carry -=1.

For example:

If the current carry|value is 0x3FF0, After one turn CCW,the carry|value (+0x4000) is 0x13FF0.

If the current carry|value is 0x3FF0, After one turn CW,the carry|value (-0x4000)is 0xFFFFFFF3FF0.

Note: The encoder value is updated regardless of whether the motor is enabled or not.The Cangaroo example is as follows:





4.2.2 Read the cumulative multi-turn encoder value

It can read the motor rotation range recorded by the encoder after power-on.

Note: Encoder single-turn value range is 0~0x4000

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	31H	CRC(32)

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2...Byte7	Byte8
01		8	code	encoder value	Checksum
			31H	value (int48_t)	CRC

value: Records the multi-turn encoder value, data type int48_t

After one turn clockwise,the value += 0x4000;

After one turn CCW,the value -= 0x4000;

For example:

If the current value is 0x3FF0, After one turn CCW,the value(+0x4000) is 0x7FF0.

If the current value is 0x3FF0, After one turn CW,the value(-0x4000) is 0xFFFFFFFFF0.

Note: When moving relative to or absolutely according to coordinate values, use the encoder value as the coordinate.

4.2.3 Read the real-time speed of the motor

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	32H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte 2...Byte 3	Byte4
01		4	code	motor speed	Checksum
			32H	rpm	CRC

rpm: Real-time motor speed, unit RPM, data type int16_t

Note: The counterclockwise rotation speed is greater than 0, and the clockwise rotation speed is less than 0.



4.2.4 Read the number of received pulses

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	33H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte 2...Byte 5	Byte6
01		6	code	pulse count	Checksum
			33H	pulses	CRC

pulses: The number of pulses received, data type int32_t

4.2.5 Read motor alarm status

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	37H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	state	Checksum
			37H	status(uint8_t)	CRC

status: Motor status, corresponding codes are shown in the table below.

Motor status	status	Remark
motor running	0	
Motor stopped	1	
Over current error	2	
Phase loss error	3	
Overvoltage error	4	
Undervoltage error	5	
Position error	6	
Encoder error	7	



4.2.6 Read position angle error

Angle error is the difference between the position angle controlled by the command and the real-time angle position of the motor. The unit is $0 \sim 51200$, which means $0 \sim 360^\circ$. For example, when the error is 1° , the value is $51200 / 360^\circ = 142.22$, and so on.

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	39H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte 2...Byte 5	Byte6
01		6	code	Angular error	Checksum
			39H	errors	CRC

errors: Real-time position error value, data type int32_t

4.2.7 Read motor enable status

When using bus control, this command can be used to obtain the enable status of the driver board.

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	3AH	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Enable state	Checksum
			3AH	status	CRC

status=01 Motor is enabled

status=00 Motor not enabled



4.2.8 Read the motor return to zero status

Read command:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	3BH	CRC(3C)

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Return to zero state	Checksum
			3BH	status	CRC

status=00 The motor has not returned to zero, or the return-to-zero process failed.

status=01 The motor has returned to zero.



4.2.9 Automatically return read-only parameters at regular intervals

This command can set parameters so that the motor can automatically return a certain read-only parameter at regular intervals, without the host computer needing to frequently issue corresponding query commands.

The command format is as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	Bytes 3-4	Byte5
01	...	5	command	code	Timer	Checksum
			01	code	times	CRC

code: The code corresponding to the read-only parameter.

times: The timeout period, in milliseconds (when times = 0, no data is returned).

Returned data:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	Byte3	Byte4
01	...	4	command	code	state	Checksum
			01	code	status	CRC

status=00 Setup failed

status=01 Set succeeded.

The parameter data format returned periodically is as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	byte n-1	byte n
01	...	n	code	Parameter value		Checksum
			code	param		CRC

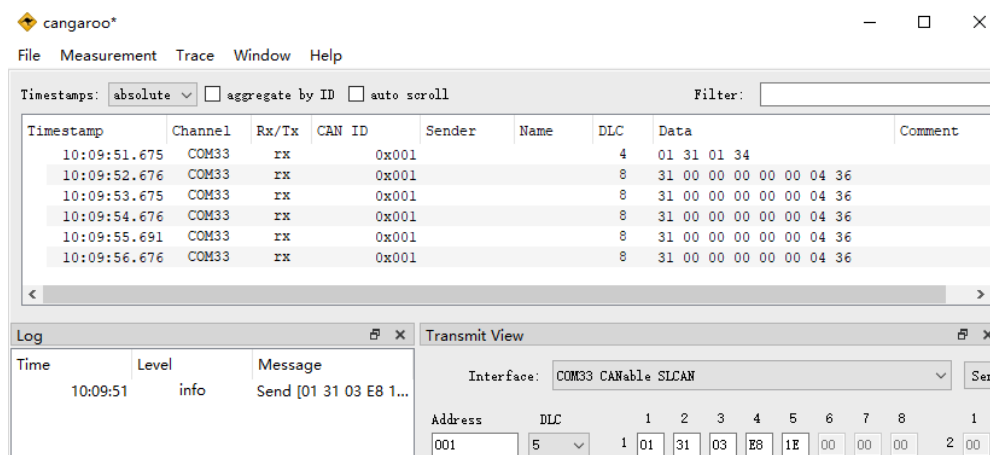
param: The corresponding read-only parameter

Taking the reading of the "accumulator multi-turn encoder value" every 1000ms as an example

Send 01 01 31 03 E8 1E

Return 01 01 31 01 34

After successful setup, the motor will return the "cumulative multi-turn encoder value" every 1000 milliseconds, as shown in the figure below.





4.2.10 Read a single configuration parameter

After writing the configuration parameters, you can use this command to read and view them.

The command format for reading write-only configuration parameters is as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	command	code	Checksum
			00	code	CRC

code: The code corresponding to the parameter

For example, to read the "working mode", the corresponding code is 82H.

The returned parameter data format is as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	byte n-1	byte n
01	...	n	code	Parameter value		Checksum
			code	param		CRC

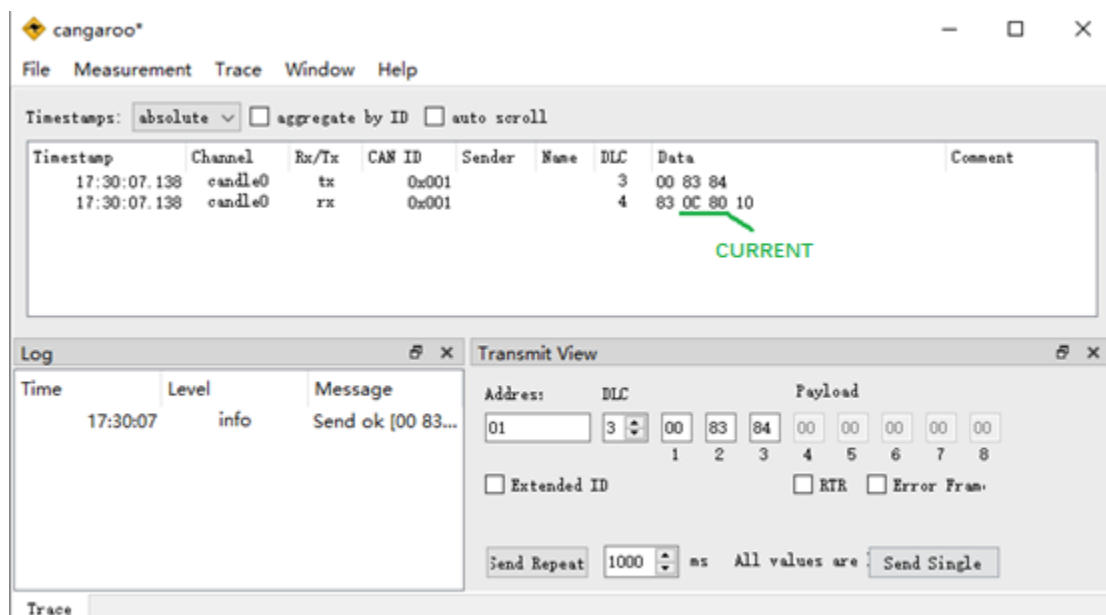
param: The corresponding configuration parameter

Note: The data format of param should be consistent with the data format when setting this parameter.

If this parameter does not support reading, the returned data will be as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	Byte3	Byte4
01	...	n	code	Parameter value		Checksum
			code	FFFFH		CRC

The following image shows an example of reading "operating current":





4.2.11 Read version information

Read hardware and firmware information.

The read command is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	40H	CRC

The returned data format is as follows:

Uplink frames (PC ← SERV0xxES)								
ID	...	DLC	Byte1	Byte 2 (b7)	Byte2 (b5-b4)	Byte2 (b3-b0)	Bytes 3-5	Byte6
01		6	code	D/E distinction	Calibration status	Hardware version	Firmware version	Checksum
			40	serials	cal	hardVer	firmVer[3]	CRC

series = 1 E series stepper motor

series = 0 D series stepper motor

cal = 1 encoder value decreases as the motor rotates CW.

cal = 2 encoder value increases as the motor rotates CW.

Firmware version: firmVer[0] = 1, firmVer[1] = 0, firmVer[2] = 0

Corresponding version V1.0.0

The hardware version correspondence is as follows:

Board type	hardVer
S42ES_RS485	1
S42ES_BUS	2
S57ES_RS485	3
S57ES_BUS	4
S28ES_RS485	5
S28ES_BUS	6
S35ES_RS485	7
S35ES_BUS	8



4.2.12 Read /Write User-defined ID

1. Write the user ID number.

The command to write the ID is as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Bytes -5	Byte6
01		6	code	Custom ID	Checksum
			42H	ID(uint32_t)	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Setting status	Checksum
			42H	status	CRC

status = 0 Write failed

status = 1 Successfully written

2. Read user ID

The command to read the ID is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	42H	CRC

Return ID data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Bytes2-5	Byte6
01		6	code	Custom ID	Checksum
			42H	ID(uint32_t)	CRC



4.3 Write-only parameter command

Note: The settings are not saved immediately after writing the instructions; the 60H instruction must be executed to save them all at once.

4.3.1 Set working mode

The instructions for setting the working mode are as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Work mode	Checksum
			82H	mode	CRC

The corresponding working modes are shown in the table below.

List of working modes		
	mode	Work mode
With encoder	00H	Pulse + Pulse Open-Loop FOC Mode
	01H	Pulse + Direction Open-Loop FOC Mode
	02H	Pulse + Pulse Closed-Loop FOC Mode
	03H	Pulse + Direction Closed-Loop FOC Mode (Default)
	04H	bus open-loop FOC mode
	05H	bus closed-loop FOC mode
No encoder	10H	Pulse + Pulse Open-Loop FOC Mode
	11H	Pulse + Direction Open-Loop FOC Mode
	14H	bus open-loop FOC mode

Note 1: An encoder is present, meaning the motor shaft has magnets. The drive board is mounted on the back and can read the encoder values.

No encoder, meaning the motor shaft has no magnets, the drive board can be installed arbitrarily, and the encoder value cannot be read.

Note 2: Pulse control mode, maximum input frequency 300kHz

Bus control mode, maximum speed 3000 RPM

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			82H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



4.3.2 Set operating current

The operating current setting instructions are as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2-3	Byte4
01	...	4	code	Current value	Checksum
			83H	current (mA)	CRC

SERV042ES maximum operating current is 3000mA (default 1600mA).

SERV057ES maximum operating current is 5200mA (default 3200mA).

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			83H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

4.3.3 Set subdivision

The detailed settings instructions are as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	subdivisions	Checksum
			84H	micstep	CRC

The MS value range (decimal) is as follows: (default value 16)

0, 2, 4, 8, 16, 32, 64, 128,

5, 10, 20, 25, 40, 50, 100, 200

Note: 0 corresponds to 256 subdivisions.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			84H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



4.3.4 Set the active level of the En pin.

The configuration command is as follows:

Downlink frames (PC → SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Set the level	Checksum
			85H	Level(uint8_t)	CRC

Level = 00 corresponds to low-level enable (L) (default).

Level = 01 corresponds to high-level enable (H).

Level = 02 corresponds to always being enabled (Hold).

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			85H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note 1: After successful setup, pulse signals will be received after a 100ms delay.

Note 2: Only valid for pulse control mode.

4.3.5 Set the motor rotation direction

The configuration command is as follows:

Downlink frames (PC → SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Set direction	Checksum
			86H	dir (uint8_t)	CRC

dir = 00 Corresponds to clockwise rotation (default)

dir = 01 Corresponding to counterclockwise rotation

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			86H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note: This command can also change the direction of motor operation controlled by the bus.



4.3.6 Set pulse delay

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			87H	delay	CRC

delay = 00 delay 0ms
delay = 01 delay 4ms
delay = 02 delay 20ms (default)
delay = 03 delay 40ms

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			87H	status	CRC

status = 0 Set failed.
status = 1 set succeeded.

4.3.7 Set bit rate

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Set protection	Checksum
			8AH	bitRate	CRC

bitRate = 00 125K
bitRate = 01 250K
bitRate = 02 500K (default)
bitRate = 03 1M

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			8AH	status	CRC

status = 0 Set failed.
status = 1 set succeeded.



4.3.8 Set slave ID

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID		DLC	Byte1	Byte2	Byte3	Byte4
01	...	4	code	Slave ID		Checksum
			8BH	ID (uint16_t)		CRC

the slave ID, ranging from 00 to 0x7FF.

Note : 00 is the broadcast ID, 01 is the default ID.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			8BH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



4.3.9 Configure slave response method

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	method	Checksum
			8CH	ackMode	CRC

ackMode = 00, no response from the slave device.

ackMode = 01, the slave device passively acknowledges, meaning it does not actively initiate data transmission.

ackMode = 02, Slave device actively responds (default value)

Example of different response methods, taking position control mode 1 as an example:

The host sends 01 FD 02 80 02 00 00 FA 00 7C

a. No response mode (ackMode = 00)

The slave device does not return any information.

b. In passive response mode (ackMode = 01)

Slave immediately returns to position control start 01 or fails 00

c. In active response mode (ackMode = 02)

Slave immediately returns to position control start 01 or fails 00

After the motor finishes running or stops when it hits the limit switch, return to 02 or 03.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			8CH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note: After setting the slave device to no response, the motor operating status can be queried using the code "F1H".



4.3.10 Configure whether to lock the axis when starting in bus mode.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Set lock shaft	Checksum
			8FH	enable	CRC

enable = 00, the shaft is not locked when starting in bus mode.

enable = 01, the shaft is locked when starting in bus mode (default).

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			8FH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



4.3.11 Set group ID

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID		DLC	Byte1	Byte2	Byte3	Byte4
01	...	4	code	Group ID		Checksum
			8DH	Group ID		CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			8DH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Application instructions for using packet IDs:

Assuming there are 6 motors, the ID settings are as follows:

	Broadcast ID	Slave ID	Group ID
Motor 1	0	1	0x 50
Motor 2	0	2	0x 50
Motor 3	0	3	0x 50
Motor 4	0	4	0x 51
Motor 5	0	5	0x 51
Motor 6	0	6	0x 51

Send 01 FD 01 2C 64 00 0C 80 1B Motor 1 Rotate 1 full circle

Send 00 FD 01 2C 64 00 0C 80 1A Motor 1-6 Rotate 1 full circle

Send 50 FD 01 2C 64 00 0C 80 6A motor 1-3 Rotate 1 full circle

Send 51 FD 01 2C 64 00 0C 80 6B motor 4-6 Rotate 1 full circle

Note: When sending commands using a packet ID, the slave device will not respond.



4.3.12 Set heartbeat protection time

Heartbeat protection refers to the system that, if the motor does not receive any instructions from the host computer within the set protection time, it will control the motor to stop urgently, preventing abnormal accidents caused by communication interruption.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Bytes2-5	Byte6
01	...	6	code	Protection time	Checksum
			89H	times	CRC

times: Protection time, in milliseconds. (Default: 0)

Note: When times=0, the heartbeat protection function is turned off.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			89H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

4.3.13 Set position to reach threshold

In bus position control mode, the "position operation completed" message will only be returned when the actual position of the motor is less than the threshold.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	Bytes3-4	Byte5
01	...	5	code	Enable	Reaching the threshold	Checksum
			98H	enable	values(uint16_t)	CRC

enable: 00 Disable position reaches threshold judgment

 01 Enable location arrival threshold judgment (default)

values: Position threshold (default value 800)

Note: The maximum value is 65535, which is 360 degrees.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	Code	Setting status	Checksum
			98H	status (uint8_t)	CRC

status = 0 Set failed.

status = 1 set succeeded.



4.3.14 Set no-verification mode

No-check mode means that after the motor receives a message, it will execute the message command without checking whether the CRC checksum is correct.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	Code	data	Checksum
			0FH	01	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			0FH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note: This instruction is for testing purposes only and cannot be saved. The motor will enter a verification mode every time it is powered on.

4.4 Stall protection instructions

If the motor fails to reach the designated position within the response time due to overload or any other reason, the stall protection will be triggered, the motor will unlock and the shaft will be released to avoid damage.

4.4.1 Set stall protection and over-tolerance settings.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID		DLC	Byte1	Byte2	Bytes 3-4	Byte5
01	...	5	code	Stall protection	Excess tolerance	Checksum
			88H	enable	value	CRC

enable = 00 disables stall protection.

enable = 01 Enables stall protection (default value)

value: Range of out-of-tolerance values (0-0x7FFF), default value 0x64

When the error exceeds the value, the stall protection is triggered, and the motor shaft is released.

The value = 0x64 corresponds to an angle of 180 degrees.

The value = 0xC8 corresponds to an angle of 360 degrees.

And so on.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			88H	status (uint8_t)	CRC

status = 0 Set failed.

status = 1 set succeeded.

After stall protection

the stall state can be released by sending a command (3DH).

the stall state can be released by controlling the motor shaft to loosen.



4.4.2 Read motor stall status

When a motor stalls, a stall flag will be set. This command can be used to determine whether a stall has occurred. If the stall protection option is enabled, the drive board will automatically shut down the driver upon a stall.

The read command is as follows:

Downlink frames (PC → SERVOxxES)				
ID	...	DLC	Byte1	Byte2
01		2	3EH	CRC

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	stalled state	Checksum
			3EH	status	CRC

status=00 Motor not stalled

status=01 The motor is stalled.

Note: When the motor stalls, the stall state can be released by writing the command (3DH).

4.4.3 Release the motor from stall state

When the motor stalls, sending this command can release the current stall state.

If a stall occurs again after the stall is cleared, the stall protection will still be triggered.

The release command is as follows:

Downlink frames (PC → SERVOxxES)				
ID	...	DLC	Byte1	Byte2
01		2	3DH	CRC

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Released status	Checksum
			3DH	status	CRC

status=00 Removal failed

status=01 Removal successful

the stall state can be released by controlling the "EN signal", that is, by controlling the motor shaft to loosen, the stall state can be released.

4.5 Save /Restore Parameters and Reset Instructions

4.5.1 Save parameter command

This command saves all modified parameters to internal storage.

The command to save parameters is as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	control word	Checksum
			60H	01	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Save status	Checksum
			60H	status	CRC

status = 0 Save failed

status = 1 Saved successfully

4.5.2 Restore factory settings

The command to restore factory settings is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01	...	2	3FH	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Restored status	Checksum
			3FH	status	CRC

status = 0 Recovery failed

status = 1 Recovery successful

Note 1 : The driver board will automatically restart after restoring factory settings.



4.5.3 Reset and restart the motor

The command to restore factory settings is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	41H	CRC

Note : This command only resets and restarts the motor; it does not modify configuration parameters.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Restored status	Checksum
			41H	status	CRC

status = 0 Reset failed

status = 1 Reset successful



Part 5. IO Port Operation Instructions

5.1 Read I/O port status

The read command is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	34H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Port status	Checksum
			34H	status	CRC

Read port status definition table

status							
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Undefined				ALM	PEND	IN_2	IN_1

PEND 1: In place; 0: Not in place

ALM 1: No alarm; 0: Alarmed

Note: 42ES does not have the IN_2 port; bit1 corresponds to the state of the En port.



5.2 Write data to IO port

The command to write a port is as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Port data	Checksum
			36H	data	CRC

Write port status definition table

data							
bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
alm_mask		pend_mask		ALM	PEND	0	0

alm_mask 0: Do not write to the ALM IO port
1: Write the ALM value to the ALM IO port
2: The ALM IO port value remains unchanged.

pend_mask 0: Do not write to the PEND IO port
1: Write the PEND value to the PEND IO port.
2: The PEND IO port value remains unchanged.

ALM ALM port write value (0/1)

PEND PEND port writes value (0/1)

Note: Writing 1 to the ALM IO port disconnects the corresponding optocoupler.

Writing 1 to the PEND IO port closes the corresponding optocoupler.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Write status	Checksum
			36H	status	CRC

status = 0 Failed to write to port

status = 1 Successfully written to port

5.3 IO port pulse frequency division output

Map the PEND port to a pulse divider output port.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID		DLC	Byte1	Byte2	Bytes 3-6	Byte7
01	...	7	code	Start level	Frequency division period	Checksum
			9FH	divLevel	divPeriod	CRC

divLevel 0: Low start level (default)

 1: High start level

divPeriod Frequency division period (default 0)

When divPeriod < 100, no frequency divider output

When divPeriod ≥ 100, the PEND port toggles once every divPeriod pulse cycle.

For example, setting 16 subdivisions, divPeriod = 3200, then the PEND port flips once for every revolution of the motor.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			9FH	status (uint8_t)	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note: To disable this feature, simply set divPeriod = 0.

Part 6. Motor return to zero instructions

There are two types of motor homing methods: "origin homing" and "coordinate homing".

6.1 Explanation of the method of returning to zero at the origin

There are two ways to "return to zero": "return to zero by origin switch" and "return to zero by mechanical limit switch".

1. Return the origin switch to zero.

Connect the origin switch to the corresponding port:

Origin switch access port	
SERVO57ES	IN_COM / IN1
SERVO42ES	IN+ / IN-

For detailed wiring instructions, please refer to 2.4 [External switch wiring method](#).

The process of returning to zero is as follows:

- The motor first searches for the switch in the set "direction" and "high speed " ;
- Upon encountering the rising edge of the switch signal, it begins to operate at "low speed " and disengages from the switch;
- After the switch is disconnected, the motor reverses direction and continues to search for the rising edge of the switch signal at "low speed " ;
- Upon encountering the rising edge of the switch signal, it then runs at "high speed " to the preset "origin offset" position and stops.
- Mark the current position as "coordinate zero point", and the origin has successfully returned to zero.

2. Mechanical limit switch returns to zero.

The zero-return torque when the mechanical limit is returned to zero needs to be set in advance via command 94H. The set torque should be sufficient to drive the load, but should not be too large to avoid damaging the equipment.

The process of returning to zero is as follows:

- searches for the mechanical limit position using the set "direction", "torque " , and "low speed " ;
- When encountering a mechanical limit switch, the rotor will stop and then run to the preset "origin offset" position to stop.
- Mark the current position as "coordinate zero point", and the origin has successfully returned to zero.



3. Single lap to zero

The "zero point" of the coordinates within a single circle needs to be set in advance via command 92H.

The direction of the single-turn zeroing needs to be set in advance via command 93H: "forward", "reverse", or "nearest".

The process of returning to zero is as follows:

- a. The motor returns to the "coordinate zero point" position within a preset single revolution at high speed in the set zero-return direction;
- b. Upon arrival, the current position is reset to zero, and the single-lap coordinates have successfully returned to zero.

6.2 Explanation of coordinate zeroing method

To return to zero directly, you need to first execute the "Return to Zero" function to determine the "zero point".

The process of returning to zero is as follows:

No search process is required; it directly and quickly reaches the "coordinate zero point" position and successfully returns to zero.



6.3 Set parameters related to zero return.

6.3.1 Set the zero-return mode, direction, and speed parameters.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)								
ID		DLC	Byte1	Byte2	Byte3	Bytes 4-5	Bytes 6-7	Byte8
01	...	8	code	Zero mode	Return to zero direction	"High speed"	"Low speed"	Checksum
			95H	HmMode	HmDir	HiSpeed	LoSpeed	CRC

Hm Mode: Sets the zero-return mode

00: Origin switch returns to zero (default value)

01: Mechanical limit switch returns to zero

02: Single lap zeroing

03: Disable the zero-reset function

Hm Dir: Sets the return-to-zero direction

00: Positive zero return (default value)

01: Reverse return to zero

02: Zero at the nearest point (only applicable to zeroing on a single lap)

HiSpeed: Sets the "high-speed" RPM for homing (range 1~3000 RPM, default = 100).

LoSpeed: Sets the "low speed" RPM for homing(range 1~100RPM, default = 10).

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			95H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



6.3.2 Set the zero-return torque and origin offset parameters.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID		DLC	Byte1	Bytes 2-3	Bytes 4-7	Byte8
01	...	8	Code	Zero-return current	Origin offset	Checksum
			96H	HmATorque (uint16_t)	Orgoffset(int32_t)	CRC

HmATorque : Sets the current when the mechanical limit switch returns to zero.

SERV042ES maximum return-to-zero current is 3000mA (default 300mA).

SERV057ES maximum return-to-zero current is 5200mA (default 600mA).

Orgoffset: Sets the offset position after returning to zero (default 0x2000).

When Orgoffset = 0, the motor returns to zero and then stops.

When Orgoffset ≠ 0, after the motor returns to zero, it will continue running until it reaches the offset position before stopping.

Note: The Orgoffset value is invalid when lapping back to zero.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			96H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



6.3.3 Configure the zero-return trigger method and timeout parameters.

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)						
ID	...	DLC	Byte1	Byte2	Bytes 3-6	Byte7
01	...	7	code	Zero-return trigger method	Zero-return timeout (ms)	Checksum
			97H	HmTrig	HmTimOut(uint32_t)	CRC

HmTrig: Sets the zero-return trigger method

00: Command triggers reset to zero (default value)

01: Automatic return to zero upon power-on (only valid in bus mode)

02: En signal triggers zero return (only valid in pulse control mode)

In pulse control mode, when hmTrig = 2, the motor is triggered to return to zero when the En signal line generates a 200ms-wide disabled level. (Pulse recognition range: 150ms~250ms)

When En is low, it enables the motor and generates a 200ms high-level signal, triggering the motor to return to zero.

When En is high, it enables the motor and generates a 200ms low-level signal, triggering the motor to return to zero.

HmTimOut: Sets the timeout period for returning to zero (default = 120000).

If the motor does not reach zero within this time, a timeout failure message will be returned.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			97H	status (uint8_t)	CRC

status = 0 Set failed.

status = 1 set succeeded.

6.3.4 Setting Zero Point Instructions

This command directly sets the current position to "zero".

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	Code	data	Checksum
			92H	00	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	status	Checksum
			92H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



6.3.5 Execute the zero-return instruction

The execution instructions are as follows:

Downlink frames (PC → SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Return to zero method	Checksum
			91H	goZeroMode	CRC

goZeroMode 00: Executes the "Return to Zero" function.

 01: Execute the "Coordinate Zeroing" function

Note 1: You can execute "Return to Zero" first to determine the "Zero Point Coordinates" before executing "Return to Zero".

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	status	Checksum
			91	status	CRC

status = 0 failed to return to zero

status = 1 start to return to zero

status = 2 return to zero complete

status = 3 times out and failed

6.4 Switch return to zero configuration example

Connect the origin switch to the corresponding port, referring to "

6.1 Explanation of the method of returning to zero at the origin"

6.4.1 Example of a switch command triggering a return to zero

The setup steps are as follows:

1. Set **working mode**
01 82 05 88
2. Set the switch **trigger level**
01 9E FF 00 FF 9D
3. Set the return-to-zero **mode**, **direction**, "High Speed", "Low Speed"
01 95 00 00 00 64 00 0A 04
4. Set **the origin offset**
01 96 02 58 00 00 20 00 11
5. Configure the zero-return **trigger method** and **timeout parameters**.
01 97 00 00 00 EA 60 E2
6. Save settings
01 60 01 62
7. Perform "return to zero"
01 9 1 00 92

It can be observed that: the motor rotates forward at high speed (100RPM) → the switch is triggered → the motor rotates in reverse at low speed (10RPM) → the motor rotates forward at low speed (10RPM) → the switch is triggered again → the motor runs at high speed to the offset position and returns to zero.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll								
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
11:08:54.706	can0	tx	0x001			3	82 05 88	
11:08:54.706	can0	rx	0x001			3	82 01 84	
11:09:08.223	can0	tx	0x001			5	9E FF 00 FF 9D	
11:09:08.223	can0	rx	0x001			3	9E 01 A0	
11:09:26.773	can0	tx	0x001			8	95 00 00 00 64 00 0A 04	
11:09:26.774	can0	rx	0x001			3	95 01 97	
11:09:42.147	can0	tx	0x001			8	96 02 58 00 00 20 00 11	
11:09:42.148	can0	rx	0x001			3	96 01 98	
11:09:58.935	can0	tx	0x001			7	97 00 00 00 EA 60 E2	
11:09:58.936	can0	rx	0x001			3	97 01 99	
11:10:11.755	can0	tx	0x001			3	60 01 62	
11:10:11.763	can0	rx	0x001			3	60 01 62	
11:10:21.646	can0	tx	0x001			3	91 00 92	
11:10:21.646	can0	rx	0x001			3	91 01 93	
11:10:35.401	can0	rx	0x001			3	91 02 94	



6.4.2 Example of automatic return to zero upon power-on

The setup steps are as follows:

1. Set **working mode**
01 82 05 88
2. Set the switch **trigger level**
01 9E FF 00 FF 9D
3. Set the return-to-zero **mode, direction, "High Speed", "Low Speed"**
01 95 00 00 00 64 00 0A 04
4. Set **the origin offset**
01 96 02 58 00 00 20 00 11
5. Configure the zero-return **trigger method** and **timeout parameters**.
01 97 01 00 00 EA 60 E3
6. Save settings
01 60 01 62

After a power outage and restart, the following can be observed: the motor rotates at high speed (100 RPM) in the forward direction → the switch is triggered → the motor rotates in the reverse direction at low speed (10 RPM) → the motor rotates in the forward direction at low speed (10 RPM) → the switch is triggered again → the motor runs at high speed to the offset position and returns to zero.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll								
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:29:28.701	candle0	tx	0x001			3	82 05 88	
14:29:28.702	candle0	rx	0x001			3	82 01 84	
14:29:43.907	candle0	tx	0x001			5	9E FF 00 FF 9D	
14:29:43.907	candle0	rx	0x001			3	9E 01 A0	
14:30:01.543	candle0	tx	0x001			8	95 00 00 00 64 00 0A 04	
14:30:01.544	candle0	rx	0x001			3	95 01 97	
14:30:20.023	candle0	tx	0x001			8	96 02 58 00 00 20 00 11	
14:30:20.024	candle0	rx	0x001			3	96 01 98	
14:30:39.856	candle0	tx	0x001			7	97 01 00 00 EA 60 E3	
14:30:39.856	candle0	rx	0x001			3	97 01 99	
14:30:49.780	candle0	tx	0x001			3	60 01 62	
14:30:49.788	candle0	rx	0x001			3	60 01 62	

Note: If the limit switch is not triggered within the set "return to zero timeout period", the "return to zero timeout failure" message will be returned.

6.5 Example of mechanical limit switch homing configuration

A suitable zero-return current needs to be set for mechanical limit switches. The zero-return current should not be too large, just enough to drive the load, to avoid damaging the mechanical device.

6.5.1 Example of mechanical limit command triggering homing

1. Set **working mode**
01 82 05 88
2. Set the return-to-zero **mode**, **direction**, "**High Speed**", "**Low Speed**"
01 95 01 00 00 64 00 0A 05
3. Set **the zero-return current** and **the origin offset**.
01 96 02 58 00 00 20 00 11
4. Configure the zero-return **trigger method** and **timeout parameters**.
01 97 00 00 00 EA 60 E2
5. Save settings
01 60 01 62
6. Perform "return to zero"
01 91 00 92

It can be observed that: the motor rotates forward at low speed (10RPM) → after touching the mechanical limit switch → the motor stops → the motor runs in reverse at high speed to the offset position, and the return to zero is completed.

Timestamps: absolute
☐ aggregate by ID
☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:32:41.315	can0	tx	0x001			3	82 05 88	
14:32:41.315	can0	rx	0x001			3	82 01 84	
14:33:11.899	can0	tx	0x001			8	95 01 00 00 64 00 0A 05	
14:33:11.899	can0	rx	0x001			3	95 01 97	
14:33:29.125	can0	tx	0x001			8	96 02 58 00 00 20 00 11	
14:33:29.126	can0	rx	0x001			3	96 01 98	
14:33:46.712	can0	tx	0x001			7	97 00 00 00 EA 60 E2	
14:33:46.713	can0	rx	0x001			3	97 01 99	
14:33:57.322	can0	tx	0x001			3	60 01 62	
14:33:57.330	can0	rx	0x001			3	60 01 62	
14:34:10.301	can0	tx	0x001			3	91 00 92	
14:34:10.302	can0	rx	0x001			3	91 01 93	
14:34:16.123	can0	rx	0x001			3	91 02 94	

Note: If the motor does not touch the mechanical limit within the set "return to zero timeout period", a "return to zero timeout failure" message will be returned.



6.5.2 Example of automatic zeroing upon power-on of mechanical limit switch

1. Set **working mode**
01 82 05 88
2. Set the return-to-zero **mode**, **direction**, "**High Speed**", "**Low Speed**"
01 95 01 00 00 64 00 0A 05
3. Set **the zero-return current** and **the origin offset**.
01 96 02 58 00 00 20 00 11
4. Configure the zero-return **trigger method** and **timeout parameters**.
01 97 01 00 00 EA 60 E3
5. Save settings
01 60 01 62

After a power outage and restart, the following can be observed: the motor rotates forward at low speed (10 RPM) → after touching the mechanical limit switch → the motor stops → the motor runs in reverse at high speed to the offset position, and the return to zero is completed.

Timestamps: ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:36:42.301	candle0	tx	0x001			3	82 05 88	
14:36:42.301	candle0	rx	0x001			3	82 01 84	
14:37:02.051	candle0	tx	0x001			8	95 01 00 00 64 00 0A 05	
14:37:02.052	candle0	rx	0x001			3	95 01 97	
14:37:17.744	candle0	tx	0x001			8	96 02 58 00 00 20 00 11	
14:37:17.745	candle0	rx	0x001			3	96 01 98	
14:37:32.271	candle0	tx	0x001			7	97 01 00 00 EA 60 E3	
14:37:32.272	candle0	rx	0x001			3	97 01 99	
14:37:42.718	candle0	tx	0x001			3	60 01 62	
14:37:42.727	candle0	rx	0x001			3	60 01 62	

Note: If the motor does not touch the mechanical limit within the set "return to zero timeout period", a "return to zero timeout failure" message will be returned.

6.6 Single-lap zero-return configuration example

To return to zero in a single lap, the "coordinate zero point" within the single lap must be set in advance using command 92H.

6.6.1 Example of single-lap command triggering zero return

First, move the motor shaft to the appropriate "zero point" position.

1. Set **working mode**
01 82 05 88
2. Set the return-to-zero **mode**, "**Nearest**", "**High Speed**", "**Low Speed**"
01 95 02 02 00 64 00 0A 08
3. Configure the zero-return **trigger method** and **timeout parameters**.
01 97 00 00 00 EA 60 E2
4. Set the "**zero point**" coordinates
01 92 00 93
5. Save settings
01 60 01 62
6. Perform "return to zero"
01 91 00 92

After saving the settings, you can move the motor shaft away from the "zero" position by turning off the power or loosening the shaft.

After executing "return to zero", it can be observed that the motor runs at high speed (100RPM) to the "zero point" position, and the return to zero is completed.

Timestamps: ☒ absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:39:10.942	candle0	tx	0x001			3	82 05 88	
14:39:10.942	candle0	rx	0x001			3	82 01 84	
14:39:32.739	candle0	tx	0x001			8	95 02 02 00 64 00 0A 08	
14:39:32.740	candle0	rx	0x001			3	95 01 97	
14:39:51.495	candle0	tx	0x001			7	97 00 00 00 EA 60 E2	
14:39:51.496	candle0	rx	0x001			3	97 01 99	
14:40:16.395	candle0	tx	0x001			3	92 00 93	
14:40:16.403	candle0	rx	0x001			3	92 01 94	
14:40:24.881	candle0	tx	0x001			3	60 01 62	
14:40:24.882	candle0	rx	0x001			3	60 01 62	
14:40:49.267	candle0	tx	0x001			3	91 00 92	
14:40:49.268	candle0	rx	0x001			3	91 01 93	
14:40:50.556	candle0	rx	0x001			3	91 02 94	



6.6.2 Example of automatic zero return upon power-on in a single revolution

First, move the motor shaft to the appropriate "zero point" position.

1. Set **working mode**

```
01      82 05 88
```

2. Set the return-to-zero mode, "Nearest", "High Speed", "Low Speed"

```
01      95 02 02 00 64 00 0A 08
```

3. Configure the zero-return trigger method and timeout parameters.

```
01      97 01 00 00 EA 60 E3
```

4. Set the "zero point" coordinates

```
01      92 00 93
```

5. Save settings

```
01      60 01 62
```

After power is cut off, move the motor shaft away from the "zero point" position.

After powering on again, it can be observed that the motor runs at high speed (100RPM) to the "zero point" position and returns to zero.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:42:17.924	can0le0	tx	0x001			3	82 05 88	
14:42:17.925	can0le0	rx	0x001			3	82 01 84	
14:42:44.654	can0le0	tx	0x001			8	95 02 02 00 64 00 0A 08	
14:42:44.654	can0le0	rx	0x001			3	95 01 97	
14:43:00.040	can0le0	tx	0x001			7	97 01 00 00 EA 60 E3	
14:43:00.041	can0le0	rx	0x001			3	97 01 99	
14:43:12.357	can0le0	tx	0x001			3	92 00 93	
14:43:12.365	can0le0	rx	0x001			3	92 01 94	
14:43:21.130	can0le0	tx	0x001			3	60 01 62	
14:43:21.131	can0le0	rx	0x001			3	60 01 62	



6.7 Example of coordinate zeroing

Before performing "coordinate zeroing", the coordinates of the "zero point" must be determined:

For switch-based or mechanical limit switch-based zeroing, "origin zeroing" must be performed first.

For the single-loop zero-return method, simply set the "zero point" coordinates.

1. Execute "Coordinate Zeroing"

01 91 01 93

Once the "zero point" coordinates are determined, regardless of the motor's position, executing "coordinate return to zero" will cause the motor to return to the zero point position at high speed.

Note: The motor must be enabled before executing "coordinate return to zero".

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:48:30.122	can0le0	tx	0x001			3	91 01 93	
14:48:30.122	can0le0	rx	0x001			3	91 01 93	
14:48:36.303	can0le0	rx	0x001			3	91 02 94	
14:48:43.054	can0le0	tx	0x001			2	31 32	
14:48:43.055	can0le0	rx	0x001			8	31 00 00 00 00 00 01 33	



Part 7. Left and right limit switch instructions

The default limit switch connection method is shown in the table below:

	Left limit port	Right limit port	Remark
SERV057ES	IN_COM / IN1	IN_COM / IN2	
SERV042ES	IN+ / IN-	EN+/EN-	Bus mode, enable right limit switch

7.1 Set limit switch parameters

The limit switch setting instructions are as follows:

The configuration command is as follows:

Downlink frames (PC → SERV0xxES)							
ID		DLC	Byte1	Byte2	Byte3	Byte4	Byte5
01	...	5	code	Limit enable	Trigger level	Limit mapping	Checksum
			9EH	EndLimit	EndLevel	reMap	CRC

EndLimit limit enable

00 : Limit switch function disabled (default value)

01 : Enable limit function

FF: Keep the original settings unchanged.

EndLevel Effective level when the End Level switch is closed

00 : Low level when the switch is closed (default value)

01 : High level when the switch is closed

FF: Keep the original settings unchanged.

reMap 00: Disable right limit port mapping (default value, only valid for 42ES)

01: Enable right limit port mapping

FF: Keep the original settings unchanged.

Limit enable description:

1. When EndLimit = 1, in bus control mode

Left limit switch triggered, motor stops moving to the left.

Right limit switch triggered, motor stops moving to the right.

2. After using the limit function for the first time or changing the limit parameters, you must perform a "return to zero" operation once .

3. The limit function is invalid in pulse control mode.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	status	Checksum
			9EH	status	CRC

status = 0 Set failed.

status = 1 set succeeded.



7.2 Example of Limit switch configuration

Connect the limit switch to the corresponding port.

1. Set **working mode**

```
01      82 05 88
```

2. Set **limit enable**, **trigger level**, and **right limit mapping**.

```
57ES Command: 01 9E 01 00 FF 9F    (No right limit mapping required)
```

```
42ES Command: 01 9E 01 00 01 A1    (Requires right limit mapping)
```

3. Set the **return-to-zero mode**, **direction**, "**High Speed**", "**Low Speed**".

```
01      95 00 00 00 64 00 0A 04
```

4. Perform "return to zero"

```
01      91 00 92
```

(After using the limit function for the first time or changing the limit switch parameters, you must perform a "zeroing" operation once.)

5. Save settings

```
01      60 01 62
```

Timestamps: ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
14:49:52.509	can0	tx	0x001			3	82 05 88	
14:49:52.510	can0	rx	0x001			3	82 01 84	
14:50:11.190	can0	tx	0x001			5	9E 01 00 FF 9F	
14:50:11.191	can0	rx	0x001			3	9E 01 A0	
14:50:30.720	can0	tx	0x001			8	95 00 00 00 64 00 0A 04	
14:50:30.721	can0	rx	0x001			3	95 01 97	
14:50:45.045	can0	tx	0x001			3	91 00 92	
14:50:45.046	can0	rx	0x001			3	91 01 93	
14:50:58.566	can0	rx	0x001			3	91 02 94	
14:51:10.545	can0	tx	0x001			3	60 01 62	
14:51:10.546	can0	rx	0x001			3	60 01 62	



7.3 Example of Left limit switch test

Controlling motor operation

01 FD 81 2C 02 70 00 00 1D

When the motor is running, triggering the left limit switch will stop the motor.

The screenshot shows the CANalyzer interface. The top section displays a list of received CAN messages:

Timestamp	Channel	Rx/Tx	CAN ID	Sender Name	DLC	Data	Comment
17:58:23.877	COM33	rx	0x001		3	FD 01 FF	
17:58:32.452	COM33	rx	0x001		3	FD 03 01	

The bottom section shows the 'Log' and 'Transmit View' tabs. The 'Log' tab displays a message at 17:58:23 with level 'info' and message 'Send [FD 81 2C 02 70 00 00 1D] to 1 o...'. The 'Transmit View' tab shows the 'Interface' set to 'COM33 CANable SLCAN'. The 'Address' field is '001' and 'DLC' is '8'. The data field is filled with the hexadecimal values: 1: FD, 2: 81, 3: 2C, 4: 02, 5: 70, 6: 00, 7: 00, 8: 1D. The 'Extended ID' checkbox is unchecked, and the 'RTR' and 'FD' checkboxes are also unchecked.

7.4 Example of Right limit switch test

Controlling motor operation

01 FD 01 2C 02 70 00 00 9D

When the motor is running, triggering the right limit switch will stop the motor.

The screenshot shows the CANalyzer interface. The top section displays a list of received CAN messages:

Timestamp	Channel	Rx/Tx	CAN ID	Sender Name	DLC	Data	Comment
18:00:57.439	COM33	rx	0x001		3	FD 01 FF	
18:01:03.794	COM33	rx	0x001		3	FD 03 01	

The bottom section shows the 'Log' and 'Transmit View' tabs. The 'Log' tab displays a message at 18:00:57 with level 'info' and message 'Send [FD 01 2C 02 70 00 00 9D] to 1 o...'. The 'Transmit View' tab shows the 'Interface' set to 'COM33 CANable SLCAN'. The 'Address' field is '001' and 'DLC' is '8'. The data field is filled with the hexadecimal values: 1: FD, 2: 01, 3: 2C, 4: 02, 5: 70, 6: 00, 7: 00, 8: 9D. The 'Extended ID' checkbox is unchecked, and the 'RTR' and 'FD' checkboxes are also unchecked.



Part 8. Bus control mode parameter description

Note 1 : The instructions in this section are only valid in bus control mode.

8.1 Description the parameters of speed and acceleration

1.speed

The speed parameter ranges from 0 to 3000. The larger the value, the faster the motor rotates.

When speed = 0, the motor stops rotating.

If the set speed is greater than the maximum speed of the control mode, the motor runs at the maximum speed of the control mode.

Note: The speed value is calibrated based on 16/32/64 subdivisions, and the speeds of other subdivisions need to be calculated based on 16 subdivisions.

For example, setting speed=1200

At 8 subdivisions, the speed is 2400 (RPM)

At 16/32/64 subdivisions, the speed is 1200 (RPM)

At 128 subdivisions, the speed is 150 (RPM)



2. acceleration

The value of the acceleration(acc) ranges from 0 to 255. The larger the value, the faster the motor accelerates/decelerates.

If acc=0, the motor runs without acceleration or deceleration, and runs directly at the set speed.

① accelerates

Suppose at time t_1 , the current speed is V_{t1} ($V_{t1} < \text{speed}$)

at time t_2 , the current speed is V_{t2}

$$t_2 - t_1 = (256 - \text{acc}) * 50 (\mu\text{S})$$

The relationship between the current speed V_{ti} , acc, and speed is as follows:

$$V_{t2} = V_{t1} + 1 \quad (V_{t2} \leq \text{speed})$$

For example: acc = 236, speed = 3000

T(ms)	speed (RPM)
0	0
1	1
2	2
3	3
...	...

T(ms)	speed (RPM)
...	...
...	...
2998	2998
2999	2999
3000	3000

② decelerates

Suppose at time t_1 , the current speed is V_{t1} ($V_{t1} > \text{speed}$)

at time t_2 , the current speed is V_{t2}

$$t_2 - t_1 = (256 - \text{acc}) * 50 (\mu\text{S})$$

The relationship between the current speed V_{ti} , acc, and speed is as follows:

$$V_{t2} = V_{t1} - 1 \quad (V_{t2} \geq \text{speed})$$



8.2 Bus control general instructions

8.2.1 Read motor operating status

The read command is as follows:

Downlink frames (PC → SERV0xxES)				
ID	...	DLC	Byte1	Byte2
01		2	F1H	CRC

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01		3	code	Running status	Checksum
			F1H	status	CRC

status: Motor operating status, corresponding codes are shown in the table below.

Motor operating status	status	Remark
Query failed	0	
Stop running	1	
Speed up	2	
slow down	3	
Run at full speed	4	
Reset to zero	5	

Note : This instruction is only valid in bus control mode.



8.2.2 Set motor enable state

In bus control mode, the motor's enable state is no longer controlled by the level of the En pin, but is controlled by the command.

The configuration command is as follows:

Downlink frames (PC → SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Work mode	Checksum
			F3H	enable	CRC

enable = 00 Motor is disabled (loose shaft).

enable = 01 Motor enable (shaft lock)

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			F3H	status	CRC

status = 0 Set failed.

status = 1 set succeeded.

Note: This instruction is only valid in bus control mode.

8.2.3 Emergency stop command for motor

The emergency stop command is as follows:

Downlink frames (PC → SERVOxxES)				
ID	...	DLC	Byte1	Byte2
01	...	2	F7H	CRC

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			F7H	status	CRC

status = 0 Emergency stop failed

status = 1 Emergency stop successful

Note: Emergency stop commands are not recommended when the motor speed exceeds 1000 RPM !

Part 9. Speed Control Mode Description

In speed control mode,

It can control the motor to run continuously at a set acceleration and speed.

It can control the motor to run at a set acceleration and speed for a set period of time.

It allows the motor to run at a set speed and acceleration as soon as it is turned on.

9.1 Speed control mode operation instructions

The execution command format is as follows:

Downlink frames (PC → SERV0xxES)									
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Byte5
01		5	code	direction	reserve	speed		acceleration	Checksum
			F6H	b7	b6-b4	b3-b0	b7-b0	acc	CRC
				dir	--	speed			

Byte2 : The highest bit indicates the direction, and the lower 4 bits, together with Byte3, indicate the speed.

Byte3 : Byte3 and the lower 4 bits of Byte2 together represent the speed.

The parameters are explained below:

dir value range 0 - 1 (CCW/CW)

speed value range 0-3000

acc values range from 0 to 255.

for example:

Send 01 F6 0 1 40 02 3A , the motor rotates forward at an acc=2 and a speed =320 (RPM).

Send 01 F6 81 40 02 BA , the motor reverses at an acc=2, speed=320 (RPM) speed.

If you want the motor to automatically stop after running for a period of time, the operating command is as follows:

Downlink frames (PC → SERV0xxES)										
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Bytes5-7	Byte8
01		8	code	dir	REV	speed		ACC	Runtime	Checksum
			F6H	b7	b6-b4	b3-b0	b7-b0	acc	runTime	CRC
				dir	—	speed				

Run time unit: 10ms , range (0~190000H)



Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Running status	Checksum
			F6H	status (uint8_t)	CRC

status = 0 Running failed

status = 1 Start running

status = 2 Running completed

status = 5 Synchronous mode has received the instruction (it will only start running after receiving the synchronization flag).

Note: This instruction is only valid in bus control mode.



9.2 stop command

The stop command is as follows:

Downlink frames (PC → SERV0xxES)									
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Byte5
01		5	code	direction	reserve	speed		acceleration	Checksum
			F6H	b7	b6-b4	b3-b0	b7-b0	acc	CRC
				0	0	0			

The stop command can control the motor to slow down and stop gradually, or it can control the motor to stop immediately. When acc is set to $\neq 0$, the motor decelerates and stops slowly. When acc is set to $= 0$, the motor stops immediately.

① Deceleration and slow stop command (acc $\neq 0$)

For example: Sending 01 F6 00 00 02 F9 will cause the motor to stop rotating at a deceleration of acc=2.

② Immediate stop command (acc = 0)

For example: Sending 01 F6 00 00 00 F7 will immediately stop the motor from rotating.

Note: It is not recommended to use the immediate stop command when the motor speed exceeds 1000 RPM !

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			F6H	status	CRC

status = 0 Speed stop failed

status = 1 Speed Stop starting...

status = 2 Speed stopped successfully

Note: You can also use the emergency stop command F7H to stop the operation.



9.3 Set automatic start command upon power-on

After setting automatic operation, the motor will automatically run in the set direction, speed, and acceleration every time the machine is turned on.

The format for setting the automatic operation command upon power-on is as follows:

Downlink frames (PC → SERV0xxES)									
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Byte5
01		5	code	direction	reserve	speed		acceleration	Checksum
			FFH	b7	b6-b4	b3-b0	b7-b0	acc	CRC
				dir	--	speed			

dir Direction value range 0-1 (default 0)
 speed speed value range 0-3000 (default 0)
 acc acceleration value range 0-255 (default 0).

Note: When speed=0, automatic operation upon power-on is disabled.

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	state	Checksum
			FFH	status	CRC

status = 0 Set failed.
 status = 1 set succeeded.



9.4 Example of Speed mode running

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62

3. Set the running direction, speed, and acceleration parameters.

01 F6 01 2C 02 26

the motor will running with the direction, speed, and acceleration.

To stop the operation, execute the stop command.

4. Stop command

01 F6 00 00 02 F9

the motor will stop at the set acceleration.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll							
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data
18:45:41.560	candle0	tx	0x001			3	82 05 88
18:45:41.561	candle0	rx	0x001			3	82 01 84
18:46:03.903	candle0	tx	0x001			3	60 01 62
18:46:03.904	candle0	rx	0x001			3	60 01 62
18:46:18.281	candle0	tx	0x001			5	F6 01 2C 02 26
18:46:18.282	candle0	rx	0x001			3	F6 01 F8
18:46:30.156	candle0	tx	0x001			5	F6 00 00 02 F9
18:46:30.157	candle0	rx	0x001			3	F6 01 F8
18:46:33.627	candle0	rx	0x001			3	F6 02 F9



9.5 Example of Automatic run mode

1. Set working mode
01 82 05 88 (Skip if already set to bus mode)
2. Set the direction, speed, and acceleration parameters.
01 FF 01 2C 02 2F
3. Save settings
01 60 01 62

After a power outage and restart, the motor will running automatically according to the set parameters.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll							
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data
18:48:54.960	candle0	tx	0x001			3	82 05 88
18:48:54.960	candle0	rx	0x001			3	82 01 84
18:49:05.271	candle0	tx	0x001			5	FF 01 2C 02 2F
18:49:05.279	candle0	rx	0x001			3	FF 01 01
18:49:14.014	candle0	tx	0x001			3	60 01 62
18:49:14.015	candle0	rx	0x001			3	60 01 62

To cancel automatic operation upon power-on, execute the following command.

1. Stop running
01 F6 00 00 02 F9
2. Clear the direction, speed, and acceleration parameters.
01 FF 00 00 00 00
3. Save settings
01 60 01 62

After a power outage and restart, the motor will not running automatically.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll							
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data
18:50:40.342	candle0	tx	0x001			5	F6 00 00 02 F9
18:50:40.342	candle0	rx	0x001			3	F6 01 F8
18:50:43.772	candle0	rx	0x001			3	F6 02 F9
18:51:00.748	candle0	tx	0x001			5	FF 00 00 00 00
18:51:00.756	candle0	rx	0x001			3	FF 01 01
18:51:51.213	candle0	tx	0x001			3	60 01 62
18:51:51.213	candle0	rx	0x001			3	60 01 62



Part 10. Position control mode description

10.1 Relative motion according to pulse number

In this mode, the motor can be controlled to run to a specified position relative to the set acceleration and speed, based on the number of pulses.

10.1.1 Relative pulse operation command

The execution command format is as follows:

Downlink frames (PC → SERV0xxES)										
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Bytes 5-7	Byte8
01		8	code	dir	reserve	speed		ACC	pulses	Checksum
			FDH	b7	b6-b4	b3-b0	b7-b0	acc	relPulses	CRC
dir	---	speed								

Byte2 : The highest bit indicates the direction, and the lower 4 bits, together with Byte3, indicate the speed.

Byte3 : Byte3 and the lower 4 bits of Byte2 together represent the speed.

The parameters are explained below:

dir Direction, value range 0 - 1 (CCW/CW)

Speed Speed, value range 0 - 3000 (RPM)

Acc Accelerator value range 0 - 255.

relPulses Pulses count value range 0 - 0xFFFFF

for example:

send 01 FD 01 40 02 00 FA 00 3B, the motor rotates 20 revolutions in the forward direction (16 microsteps) with acc=2 and speed=0x140;

send 01 FD 81 40 02 00 FA 00 BB, the motor rotates in the opposite direction for 20 revolutions (16 microsteps) with acc=2 and speed=0x140;

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Running status	Checksum
			FDH	status (uint8_t)	CRC

The operation failed at position status = 0.

status = 1 Start running

status = 2 Position completed

status = 3 Stopped upon touching limit switch

status = 5 Synchronous mode has received the instruction (it will only start running after receiving the synchronization flag).

Note: You can set whether to return to the running state using the command " 8CH ".



10.1.2 Stop command

The stop command is as follows:

Downlink frames (PC → SERVOxxES)										
ID	...	DLC	Byte1	Byte2			Byte3	Byte4	Bytes 5-7	Byte8
01		8	code	dir	reserve	speed		ACC	pulses	Checksum
			FDH	b7	b6-b4	b3-b0	b7-b0	acc	0	CRC
				0	0	0				

The stop command can control the motor to slow down and stop gradually, or it can control the motor to stop immediately. When acc is set to $\neq 0$, the motor decelerates and stops slowly. When acc is set to 0, the motor stops immediately.

① Slow down and stop command (acc $\neq 0$)

for example:

Sending 01 FD 0 0 00 04 00 00 00 02

The motor will stop rotating at a deceleration of acc=4.

② Immediate stop command (acc = 0)

for example:

Sending 01 FD 00 00 00 00 00 00 FE

The motor will stop immediately.

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			FDH	status (uint8_t)	CRC

Status = 0 Stop failed

status = 1 Stop Starting...

status = 2 position stopped completing

status = 3 Stopped upon touching limit switch

Note: You can also use the emergency stop command F7H to stop the operation.



10.1.3 Example of Relative Pulse Operation

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62 (Skip if already set to bus mode)

3. Set the running direction, speed, and relative pulse count.

01 FD 01 2C 02 04 E2 00 13

(speed = 300 RPM, acceleration = 2, 100 revolutions clockwise)

4. If you need to slow down and stop the machine during operation, the command is as follows:

01 FD 00 00 02 00 00 00 00

5. If an emergency stop is required during operation, the command is as follows:

01 F7 F8

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll								
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
15:16:18.425	candle0	tx	0x001			3	82 05 88	
15:16:18.426	candle0	rx	0x001			3	82 01 84	
15:16:28.333	candle0	tx	0x001			3	60 01 62	
15:16:28.333	candle0	rx	0x001			3	60 01 62	
15:16:47.925	candle0	tx	0x001			8	FD 01 2C 02 04 E2 00 13	
15:16:47.925	candle0	rx	0x001			3	FD 01 FF	
15:17:11.273	candle0	rx	0x001			3	FD 02 00	



10.2 Absolute motion based on pulse count

In this mode, the motor can be controlled to run to the specified position with a set acceleration and speed, based on the number of pulses.

Note: Before using absolute motion, a zeroing process must be performed once (instruction 0x9 1 or 0x9 2) to mark the "zero point".

10.2.1 Absolute Pulse Operation Command

The execution command format is as follows:

Downlink frames (PC → SERV0xxES)								
ID		DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	Pulses	Checksum
			FEH	speed		acc	absPulses	CRC

The parameters are explained below:

Speed speed value range 0 - 3000 (RPM)

Acc accelerator value range 0 - 255.

absPulses absolute number of pulses

value range of int24_t (-8388607, +8388607).

For example, the current pulse number is any value.

Send 01 FE 02 58 02 00 40 00 9B

The motor runs at a speed of 600 RPM, with an acceleration of 2, and an absolute speed of 0x4000 .

That is, after the motor finishes running, the current pulse count becomes 0x4000 .

For example, the current coordinates are any value

Send 01 FE 02 58 02 FF C0 00 1A

The motor runs at a speed of 600 RPM, with an acceleration of 2, and an absolute speed of -0x4000 .

That is, after the motor finishes running, the current pulse count becomes -0x4000 .



Returned data:

Message (Host computer ← Driver board)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			F E	status (uint8_t)	CRC

The operation failed at position status = 0.

status = 1, Start running

status = 2 Position completed

status = 3 Stopped upon touching limit switch

status = 5 Synchronous mode has received the instruction (it will only start running after receiving the synchronization flag).

Note: You can set whether to return to the running state using the command " 8CH ".



10.2.2 Stop command

The stop command is as follows:

Downlink frames (PC → SERV0xxES)								
ID	...	DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	pulses	Checksum
			FEH	0		acc	0	CRC

The stop command can control the motor to slow down and stop gradually, or it can control the motor to stop immediately.

When acc is set to $\neq 0$, the motor slows down and stops gradually.

When acc is set to $= 0$, the motor stops immediately.

① Slow down and stop command (acc $\neq 0$)

for example:

Sending 01 FE 0 0 00 04 00 00 00 03

The motor will stop rotating at a deceleration of acc=4.

② Immediate stop command (acc = 0)

for example:

Sending 01 FE 0 0 00 00 00 00 00 FF

The motor will stop immediately.

Note: It is not recommended to use the immediate stop command when the motor speed exceeds 1000 RPM !

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			FE	status (uint8_t)	CRC

Status = 0

Stop failed

status = 1

Position stop starting...

status = 2

position stopped completing

status = 3

Stopped upon touching limit switch

Note: You can also use the emergency stop command F7H to stop the operation.



10.2.3 Example of Absolute Pulse Operation

When operating in absolute pulse mode, the "zero point" pulse position needs to be set first. There are two ways to set it:

1. "Return to Zero" setting;
2. Setting the "92H command mode";

If not set, the default power-on location is "midnight".

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62 (Skip if already set to bus mode)

3. Set zero point

01 92 00 93

4. Set the running speed and absolute pulse count.

01 FE 01 2C 02 01 00 00 2F

(Run at speed = 300 RPM , acc = 2, to pulse position 0x10000)

5. After the process is complete, you can view the number of pulses.

01 33 34

6. If you need to slow down and stop the machine during operation, the command is as follows:

01 FE 00 00 02 00 00 00 01

7. If an emergency stop is required during operation, the command is as follows:

01 F7 F8

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll								
Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
15:18:11.130	candle0	tx	0x001			3	82 05 88	
15:18:11.130	candle0	rx	0x001			3	82 01 84	
15:18:31.130	candle0	tx	0x001			3	60 01 62	
15:18:31.131	candle0	rx	0x001			3	60 01 62	
15:18:41.544	candle0	tx	0x001			3	92 00 93	
15:18:41.545	candle0	rx	0x001			3	92 01 94	
15:19:06.908	candle0	tx	0x001			8	FE 01 2C 02 01 00 00 2F	
15:19:06.909	candle0	rx	0x001			3	FE 01 00	
15:19:14.413	candle0	rx	0x001			3	FE 02 01	
15:19:24.103	candle0	tx	0x001			2	33 34	
15:19:24.103	candle0	rx	0x001			6	33 00 01 00 00 35	



10.3 Relative motion according to coordinate values

In this mode, the motor can be controlled to move to a specified position relative to the coordinate values at a set acceleration and speed.

Note: The coordinate values are the cumulative multi-turn encoder values (16384/turn), read using the command "31 H".

10.3.1 Relative coordinate running instructions

The execution command format is as follows:

Downlink frames (PC → SERV0xxES)								
ID		DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	relative coordinates	Checksum
			F4H	speed		acc	relAxis	CRC

The parameters are explained below:

Speed speed value range 0 - 3000 (RPM)

Acc accelerator value range 0 - 255.

relAxis relative coordinate,
value range of int24_t (-8388607 , +8388607).

For example, the current coordinate is 0x8000

Send 01 F4 02 58 02 00 40 00 91

The motor operates at a speed of 600 RPM, an acceleration of 2, and a relative speed of 0x4000 .

That is, after the motor finishes running, the current coordinate becomes 0xC000 .

For example, the current coordinate is 0x8000

Send 01 F4 02 58 02 FF C0 00 09

The motor operates at a speed of 600 RPM, an acceleration of 2, and a relative speed of -0x4000 .

That is, after the motor finishes running, the current coordinate becomes 0x4000 .

Note: The "31H" command can read coordinate values.



Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Running status	Checksum
			F4H	status	CRC

status = 0 start failed.

status = 1 start running

status = 2 position completed

status = 3 Stopped upon touching limit switch

status = 5 Synchronous mode has received the instruction (it will only start running after receiving the synchronization flag).



10.3.2 Stop command

The stop command is as follows:

Downlink frames (PC → SERV0xxES)								
ID	...	DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	relative coordinates	Checksum
			F4H	0		acc	0	CRC

The stop command can control the motor to slow down and stop gradually, or it can control the motor to stop immediately. When acc is set to $\neq 0$, the motor slows down and stops gradually.

When acc is set to $= 0$, the motor stops immediately.

① Slow down and stop command (acc $\neq 0$)

for example:

Send 01 F 4 0 0 00 04 00 00 00 F9

The motor will stop rotating at a deceleration of acc=4.

② Immediate stop command (acc = 0)

for example:

Send 01 F 4 0 0 00 00 00 00 00 F5

The motor will stop immediately.

Note: It is not recommended to use the immediate stop command when the motor speed exceeds 1000RPM !

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			F4	status (uint8_t)	CRC

Status = 0 Stop failed

status = 1 Stop Startign...

status = 2 position stopped completing

status = 3 Stopped upon touching limit switch

Note: You can also use the emergency stop command F7H to stop the operation.



10.3.3 Example of relative coordinate operation

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62 (Skip if already set to bus mode)

3. Set the running direction, speed, and relative coordinates.

01 F4 01 2C 02 02 80 00 A6

(speed = 300RPM , acc= 2, relative to coordinates 0x28000)

4. After the process is complete, you can view the coordinates.

01 31 32

5. If you need to slow down and stop the machine during operation, the command is as follows:

01 F4 00 00 02 00 00 00 F7

6. If an emergency stop is required during operation, the command is as follows:

01 F7 F8

Timestamps: ☒ absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
15:23:28.004	candle0	tx	0x001			3	82 05 88	
15:23:28.004	candle0	rx	0x001			3	82 01 84	
15:23:37.096	candle0	tx	0x001			3	60 01 62	
15:23:37.097	candle0	rx	0x001			3	60 01 62	
15:23:58.074	candle0	tx	0x001			8	F4 01 2C 02 02 80 00 A6	
15:23:58.075	candle0	rx	0x001			3	F4 01 F6	
15:24:03.250	candle0	rx	0x001			3	F4 02 F7	
15:24:12.429	candle0	tx	0x001			2	31 32	
15:24:12.430	candle0	rx	0x001			8	31 00 00 00 02 80 03 B7	



10.4 Absolute motion based on coordinate values

In this mode, the motor can be controlled to run to the specified position with a set acceleration and speed, based on the coordinate values.

Note 1: The coordinate values are the cumulative multi-turn encoder values (16384/turn), read using the command "31H".

Note 2 : Supports real-time updates of speed and coordinates, meaning that a new command can be issued to change the speed and coordinates while the previous command is running.

Note 3: Before using absolute motion, a zeroing process must be performed once (instruction 0x91 or 0x92) to mark the "zero point".

10.4.1 absolute coordinate execution command

The execution command format is as follows:

Downlink frames (PC → SERV0xxES)								
ID		DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	relative coordinates	Checksum
			F5H	speed		acc	abs Axis	CRC

The parameters are explained below:

Speed speed value range 0 - 3000 (RPM)

Acc accelerator value range 0 - 255.

absAxis absolute coordinate
value range of int24_t(-8388607 , +8388607).

For example, the current coordinates are any value

Send 01 F5 02 58 02 00 40 00 92

The motor runs at a speed of 600 RPM, with an acceleration of 2, and an absolute speed of 0x4000 .

That is, after the motor finishes running, the current coordinate becomes 0x4000 .

For example, the current coordinates are any value

Send 01 F5 02 58 02 FF C0 00 11

The motor runs at a speed of 600 RPM, with an acceleration of 2, and an absolute speed of -0x4000 .

That is, after the motor finishes running, the current coordinate becomes -0x4000 .

Note: The "31H" command can read coordinate values.



Returned data:

Uplink frames (PC ← SERV0xxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Running status	Checksum
			F5	status	CRC

status = 0 Start failed.

status = 1 Start running...

status = 2 Position completed

status = 3 Stopped upon touching limit switch

status = 5 Synchronous mode has received the instruction (it will only start running after receiving the synchronization flag).



10.4.2 Stop command

The stop command is as follows:

Downlink frames (PC → SERV0xxES)								
ID		DLC	Byte1	Byte2	Byte3	Byte4	Bytes 5-7	Byte8
01	...	8	code	speed		acceleration	relative coordinates	Checksum
			F5H	0		acc	0	CRC

The stop command can control the motor to slow down and stop gradually, or it can control the motor to stop immediately. When acc is set to $\neq 0$, the motor slows down and stops gradually. When acc is set to $= 0$, the motor stops immediately.

① Deceleration and slow stop command (acc $\neq 0$)

for example:

Send 01 F 5 0 0 00 04 00 00 00 FA.

The motor will stop rotating at a deceleration of acc=4.

② Immediate stop command (acc = 0)

for example:

Sending 01 F 5 0 0 00 00 00 00 00 F6

The motor will stop immediately.

Note: It is not recommended to use the immediate stop command when the motor speed exceeds 1000 RPM !

Returned data:

Uplink frames (PC ← SERV0xxES)					
ID	...	DLC	Byte1	Byte2	Byte3
01	...	3	code	Stopped state	Checksum
			F5H	status (uint8_t)	CRC

Status = 0 Stop failed.

status = 1 position stop start...

status = 2 position stopped completing

status = 3 Stopped upon touching limit switch

Note: You can also use the emergency stop command F7H to stop the operation.



10.4.3 Example of absolute coordinates operation

You need to first set the "zero point" coordinates. There are two ways to do this:

1. "Return to Zero" setting;
2. Setting the "92H command mode";

If not set, the default power-on position is the "zero point" coordinate.

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62 (Skip if already set to bus mode)

3. Set zero point

01 92 00 93

4. Set running speed, absolute coordinates

01 F5 01 2C 02 02 80 00 A7

(Run at speed = 300RPM, acc = 2, to coordinates 0x28000)

5. After the process is complete, you can view the coordinates.

01 31 32

6. If you need to slow down and stop the machine during operation, the command is as follows:

01 F5 00 00 02 00 00 00 F8

7. If an emergency stop is required during operation, the command is as follows:

01 F7 F8

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
15:30:26.779	candle0	tx	0x001			3	82 05 88	
15:30:26.780	candle0	rx	0x001			3	82 01 84	
15:30:37.838	candle0	tx	0x001			3	60 01 62	
15:30:37.839	candle0	rx	0x001			3	60 01 62	
15:30:46.419	candle0	tx	0x001			3	92 00 93	
15:30:46.420	candle0	rx	0x001			3	92 01 94	
15:31:03.487	candle0	tx	0x001			8	F5 01 2C 02 02 80 00 A7	
15:31:03.488	candle0	rx	0x001			3	F5 01 F7	
15:31:08.688	candle0	rx	0x001			3	F5 02 F8	
15:31:13.932	candle0	tx	0x001			2	31 32	
15:31:13.934	candle0	rx	0x001			8	31 00 00 00 02 7F FE B1	



10.4.4 Example of Real-time updates operation

This mode supports real-time updates of speed and coordinates.

You need to first set the "zero point" coordinates. There are two ways to do this:

1. "Return to Zero" setting;
2. Setting the "92H command mode";

If not set, the default power-on position is the "zero point" coordinate.

1. Set working mode

01 82 05 88 (Skip if already set to bus mode)

2. Save settings

01 60 01 62 (Skip if already set to bus mode)

3. Set zero point

01 92 00 93

4. Set running speed, absolute coordinates

01 F5 01 2C 02 7F 80 00 24

(Run at speed = 300RPM, acc = 2, to pulse position 0x7F8000)

Waiting about 20 seconds for the motor to run, then update the speed and coordinate values.

5. Update the running speed, absolute coordinates

01 F5 02 58 02 02 80 00 D4

(Run at speed = 600RPM, acc = 2, to pulse position 0x28000)

The motor will change speed and target position during operation.

6. After the process is complete, you can view the coordinates.

01 31 32

Timestamps: ☒ absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
15:39:31.224	candle0	tx	0x001			3	82 05 88	
15:39:31.225	candle0	rx	0x001			3	82 01 84	
15:39:40.102	candle0	tx	0x001			3	60 01 62	
15:39:40.103	candle0	rx	0x001			3	60 01 62	
15:39:48.314	candle0	tx	0x001			3	92 00 93	
15:39:48.315	candle0	rx	0x001			3	92 01 94	
14:30:42.390	candle0	tx	0x001			8	F5 01 2C 02 7F 80 00 24	
14:30:42.391	candle0	rx	0x001			3	F5 01 F7	
14:31:02.958	candle0	tx	0x001			8	F5 02 58 02 02 80 00 D4	
14:31:02.959	candle0	rx	0x001			3	F5 01 F7	
14:31:22.977	candle0	rx	0x001			3	F5 02 F8	
14:31:27.428	candle0	tx	0x001			2	31 32	
14:31:27.429	candle0	rx	0x001			8	31 00 00 00 02 80 01 B5	



Part 11. Multi- motor synchronous motion description

Synchronous motion refers to starting multiple motors connected to a single bus to move simultaneously. There are two ways to achieve this:

Method 1:

Sending a command using broadcast ID 0 will cause all motors on the bus to execute the command, which is suitable for all motors on the bus to execute the same command.

Method 2:

When a command is sent using a group address, all motors in the same group on the bus will execute it. This is suitable for group execution of commands by motors on the bus.

Method 3:

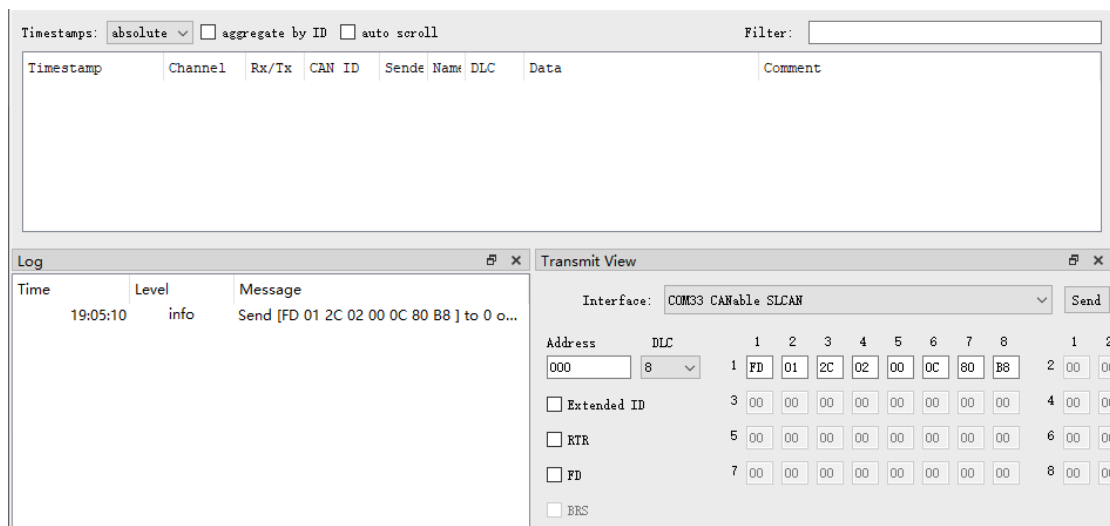
The multi-motor synchronization flag function is used to enable different motors on the bus to execute different motion commands. This method allows for the control of an unlimited number of motors.

11.1 synchronous motion mode 1 - Broadcast

To send motor movement commands using broadcast ID 0, such as instructing all motors on the bus to move relative to each other for 3200 pulses, the following command can be sent.

00 FD 01 2C 02 00 0C 80 B8

All motors on the bus will move relative to each other for 3200 pulse positions.



11.2 synchronous motion mode 2 - grouping method

To send motor movement commands using group addresses, such as instructing all motors with group address 0x50 on the bus to run relative to each other for 3200 pulses, the following command can be sent.

```
50      FD 01 2C 02 00 0C 80 08
```

All motors with group address 0x50 on the bus will run for 3200 pulse positions.

See the section 4.3.11 for group address settings .

11.3 synchronous motion mode 3 - Synchronization mark

After using the synchronization flag function, the motor will not execute the motion command (F4/F5/F6/FD/FE) immediately. It will wait for the synchronization execution command to be received before it starts executing.

11.3.1 Command to enable/disable synchronization function

The configuration command is as follows:

Downlink frames (PC → SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			4AH	enable	CRC

enable = 00 disables the multi-motor synchronization flag function (default value).

enable = 01 Enables the multi-motor synchronization flag function.

Returned data:

Uplink frames (PC ← SERVOxxES)					
ID		DLC	Byte1	Byte2	Byte3
01	...	3	code	Setting status	Checksum
			4AH	status (uint8_t)	CRC

status = 0 Set failed.

status = 1 set succeeded.



11.3.2 Synchronous execution instructions

The motor only begins to execute the previously received motion commands after receiving the synchronization execution command.

Synchronous execution instructions use a broadcast ID, so all motors on the bus can receive them.

The synchronous execution instructions are as follows:

Downlink frames (PC → SERV0xxES)				
ID		DLC	Byte1	Byte2
00	...	2	code	Checksum
			4BH	CRC

Returned data: None.

If there are many motors on the bus, some motors may not receive the synchronization execution command due to interference or other reasons. In this case, consider sending the synchronization execution command repeatedly at intervals of about 1ms.



11.3.3 Example of multi-motor synchronous operation

Taking two motors on the bus as an example, with ID=01 and ID=02 respectively.

1. Configure the operating mode (skip this step if it's already set to bus mode).

```
00      82 05 87
```

2. Configure multi-motor synchronization (skip this step if synchronization is already enabled).

```
00      4A 01 4B
```

3. Save the settings (skip this step if you don't need to save the settings).

```
00      60 01 62
```

4. Motor 1 setting: Speed mode, forward rotation, speed 640 RPM, acceleration 2

```
01      F6 02 80 02 7B
```

5. Motor 2 settings: relative pulse mode, reverse, speed 100 RPM, acceleration 2, 320000 pulses.

```
02      FD 80 64 02 04 E2 00 CB
```

6. Send synchronous execution command

```
00      4B 4B
```

...

(Whether it is necessary to send synchronization commands multiple times at 1ms intervals depends on the situation)

The motors will begin to run synchronously according to their configuration parameters.

Timestamps: absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
16:27:00.415	can0	tx	0x000			3	82 05 87	
16:27:10.786	can0	tx	0x000			3	4A 01 4B	
16:27:19.118	can0	tx	0x000			3	60 01 62	
16:27:36.656	can0	tx	0x001			5	F6 02 80 02 7B	
16:27:36.656	can0	rx	0x001			3	F6 05 FC	
16:28:02.781	can0	tx	0x002			8	FD 80 64 02 04 E2 00 CB	
16:28:02.781	can0	rx	0x002			3	FD 05 04	
16:28:13.272	can0	tx	0x000			2	4B 4B	

Part 12. CANGAROO usage examples

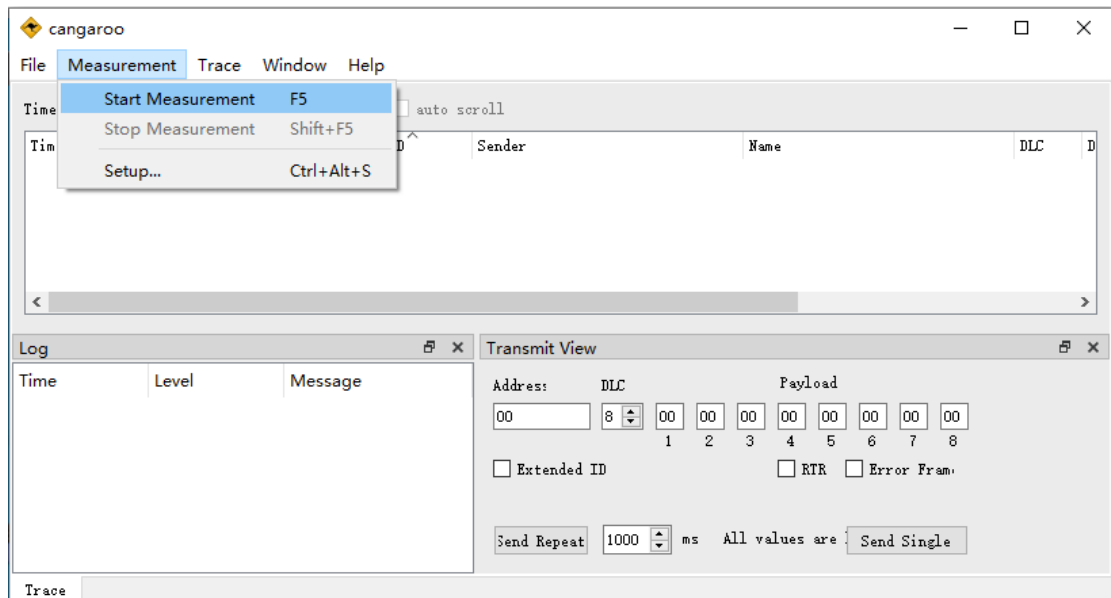
The following example uses the "cangaroo" host computer software and the "MKS CANable" USB to CAN module.

12.1 Motor parameter configuration

1. Select control mode: Mode -> 05 CAN Bus Closed-Loop FOC Mode
2. Set the baud rate: CanRate -> 500K (default value)
3. Set the slave ID: CanID -> 01 (default value)

12.2 cangaroo parameter configuration

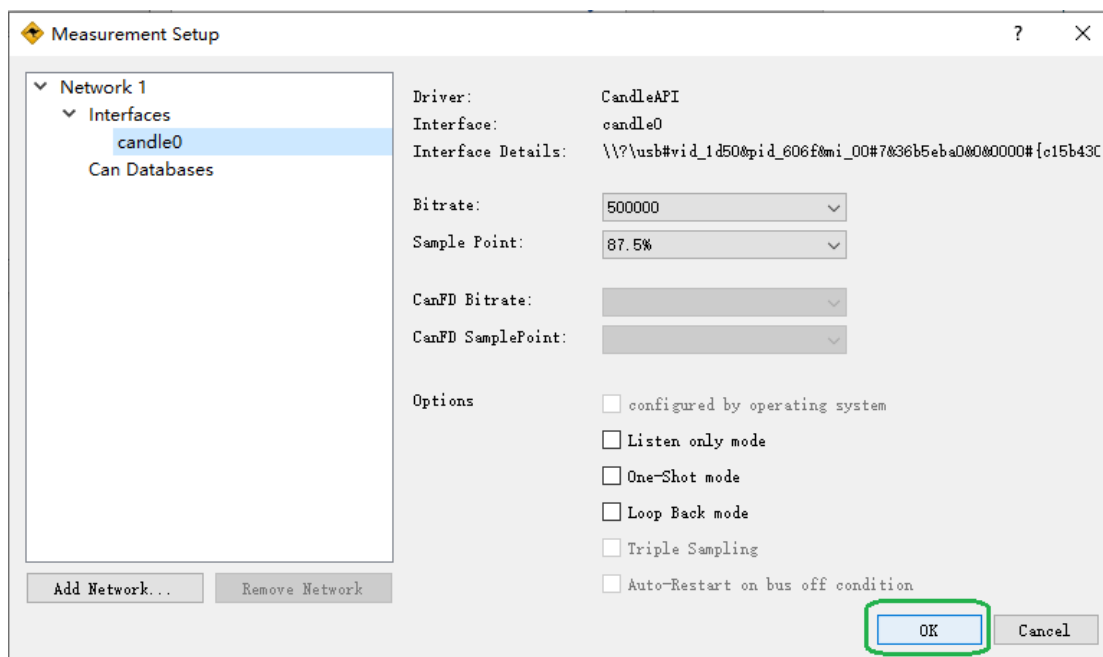
1. Double-click "cangaroo.exe" to run the host computer software;
2. In the Cangaroo window, select the menu " Measurement " - > " Start Measurement ", as shown in the image below.



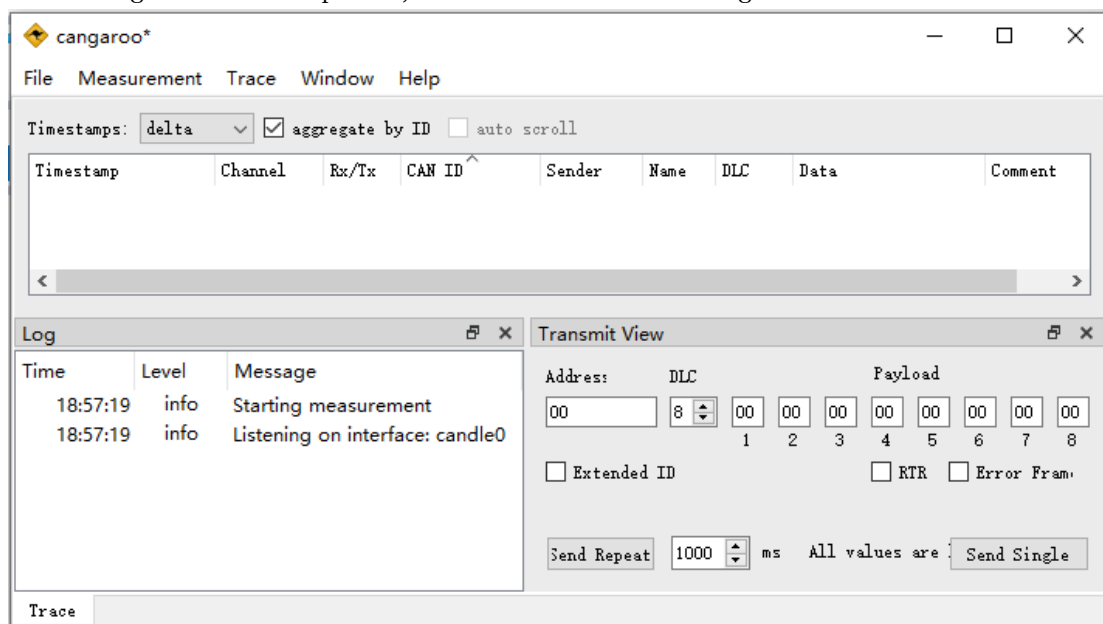
3. In the pop-up Measurement Setup window, click " candle0 ", as shown in the image below.



4. Use the default parameters without making any changes, and click "OK ", as shown in the image below.

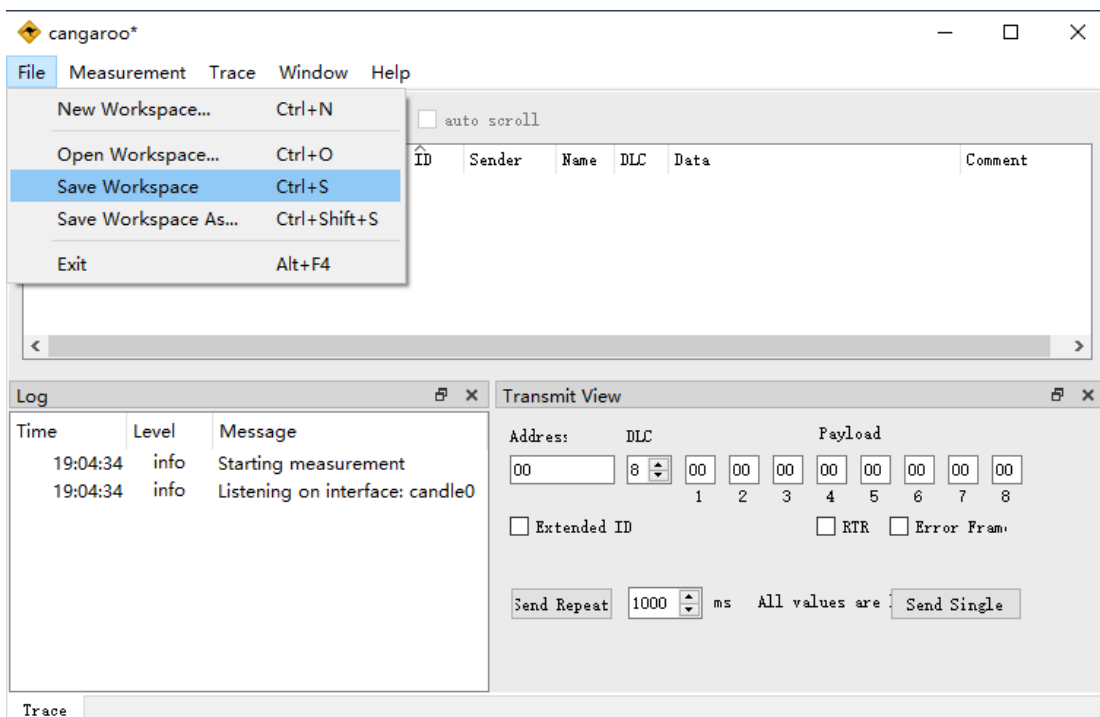


5. Configuration complete, as shown in the image below.

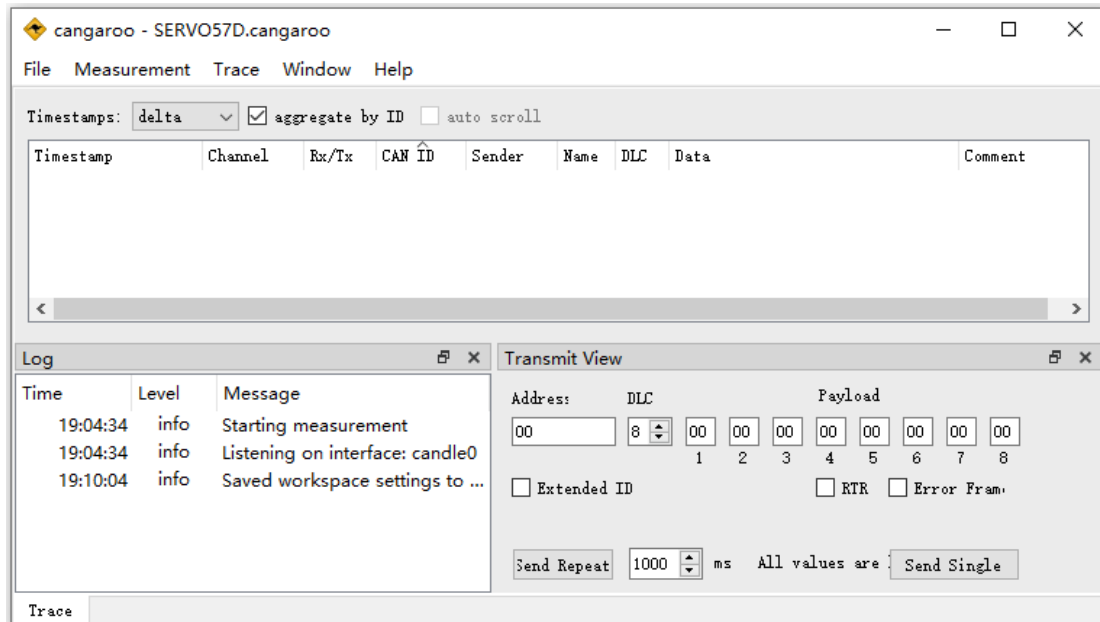




6. Select "File - > " Save Workspace " from the menu, choose the save path and name, and save the configuration.



7. After saving, it will look like the image below.





12.3 Example of reading encoder values

Send 01 31 32 to read encoder value.

Return 01 31 00 00 00 00 03 35

As shown in the figure below

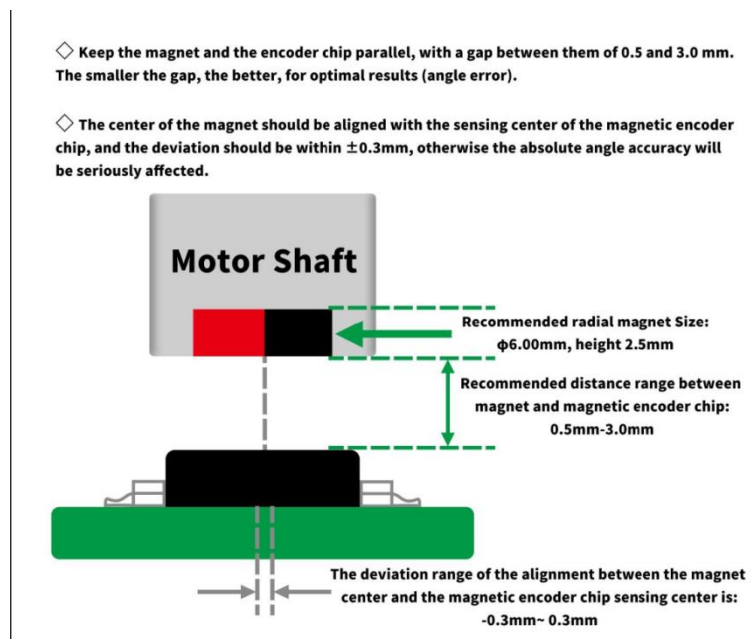
Timestamps: absolute ☐ aggregate by ID ☐ auto scroll

Timestamp	Channel	Rx/Tx	CAN ID	Sender	Name	DLC	Data	Comment
16:31:20.482	candle0	tx	0x001			2	31 32	
16:31:20.483	candle0	rx	0x001			8	31 00 00 00 00 00 03 35	

Part 13. Frequently Asked Questions and Precautions

13.1 Precautions

1. Power input voltage: 20V - 48V;
2. Do not plug or unplug the power cord or signal cable while the circuit is powered on, as this may damage the driver board.
3. The A+, A-, B+, and B- phase lines must correspond; otherwise, an error will occur.
4. We recommend that users purchase our matching motors directly to avoid the risk of incompatibility. If you choose to use a custom motor in closed-loop mode instead of our motors, the following conditions must be met:
 - (1) The motor step distance is 1.8 degrees.
 - (2) The motor's internal resistance is less than 10 ohms.
 - (3) A radial magnet can be installed on the back of the motor.
 - (4) When installing magnets, please note the following diagram:



5. After the magnets are installed, a calibration procedure must be performed to compensate for any installation deviations of the magnets; refer to section 4.1.



13.2 Frequently Asked Questions

Serial Number	question	Solution
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
1 2		

Part 14. contact us

<https://makerbase.aliexpress.com/>

<https://www.youtube.com/channel/UC2i5I1tcOXRJ2ZJiRxwpCUQ>

<https://github.com/makerbase-motor>